Assembler 32 Bit Memory Mapping

Training Manual (Based on OS390 systems)

Authored by: Neeraj Aggarwal
Emp: 140333
Date : June 17, 2011

Purpose:

This training manual is developed by **Neeraj Aggarwal** (140333) as part of Infosys' capability-building initiative for legacy modernization programs.

While leading multiple modernization programs, 'separation' efforts for Mission Critical initiatives in Core Banking, Card Domain, Financial Products & airline domain involving TPF, assembler, and core banking systems, it is observed that many team members lacked structured, practical guidance on assembler fundamentals, which created bottlenecks in delivery and quality.

To address this gap, creating this comprehensive training guide covering:

- DSECTs and memory layout
- Boundary alignment
- ORG/EQU usage
- Packed decimal operations
- Machine code translation
- Instruction-level behavior
- Practical desk-assembly exercises

This material will be used to train Mission-Critical systems, modernization teams across multiple projects and became a foundational reference for new engineers working on high-complexity legacy systems.

Contributions to This Training Material

- Designed the full curriculum for assembler and TPF fundamentals
- Authored explanations, diagrams, and examples tailored to Infosys modernization projects
- Created hands-on exercises based on real banking system scenarios
- Simplified complex assembler concepts for new engineers
- Delivered multiple training sessions across teams
- Updated the content based on feedback and evolving project needs

Business Impact

This training manual will directly support:

- Faster onboarding of engineers into core banking modernization
- Reduction in assembler-related defects
- Improved delivery timelines for legacy transformation programs
- Increased team confidence in handling TPF and assembler components
- Standardization of assembler coding practices across teams

Usage and Adoption

- To be used across multiple Infosys Financial Services accounts
- To be adopted by cross-functional teams working on modernization
- To serve as a reference guide for new hires and lateral engineers
- Recognized by delivery leadership for improving capability maturity

ASSEMBLER INSTRUCTIONS

Contents

DSECT - Dummy section or data section

DSECTs are defined to avoid 'base-register and displacement'-coding and to make the source code more readable.

Example:

Address
L’80

Name & First Name	Street & Nbr	P-Code & Cityname
L’30	L’30	L’20

Name		1st Name	Street	Nbr	P-Code	Cityname
L’14	2	L’14	L’26	L’4	L’4	L’16

```
06A     LAST        DS      H       label of the previous DSECT
06C     ADDRESS     DSECT           relative address is set to 000
000     ADDR        DS      0CL80
000     NA          DS      0CL30
000     NAME        DS      CL14
00E                 DS      CL2
010     FNAME       DS      CL14
01E     STR         DS      0CL30
01E     STREET      DS      CL26
038     NBR         DS      CL4
03C     CITY        DS      0CL20
03C     PCODE       DS      CL4
040     CITYNAME    DS      CL16
050     $IS$        CSECT           relative addr. is reset to previous relative addr.
06A     NEW         DS      CL2     label of the previous DSECT
```

- **a DSECT is a layout-definition of one or more a logical records (LRECs / Datasets).**
- **no storage is allocated !!!**
- **the end of a DSECT is marked by one of the macros CSECT, DSECT, END or FINIS.**
- **each used DSECT in a program has its own defined base-register done with USING, for example: USING R4,ADDRESS.**

Definition Types (Formats)

C Character format (EBCDIC, Extended Binary Coded - Decimal Interchange Code):
1 EBCDIC-character in 1 byte

X Hexadecimal format: 2 hex-digits in 1 byte

F Fullword format: Signed decimal number in a 4-byte field (on boundary)

H Halfword format: Signed decimal number in a 2-byte field (on boundary)

D Doubleword format: Signed decimal number in an 8-byte field (on boundary)
Note: No DC possible with this type!

A Address-Constant format: Address-value in a 4-byte field (on boundary)

B Binary format: 8 binary digits in 1 byte

P Packed Decimal format: 2 decimal digits in 1 byte (signed)

Boundaries

There are 3 types of boundaries:

- Doubleword boundary (relative address is divisible by 8)
- Fullword boundary (relative address is divisible by 4)
- Halfword boundary (relative address is divisible by 2)

ORG - Organize

```
         ADDRESS                DSECT
000      ADDR                   DS       CL80
050                             ORG      ADDR
000      NAME                   DS       CL30
01E                             ORG      NAME
000      LNAME                  DS       CL14
00E                             DS       CL2
010      FNAME                  DS       CL14
01E      STREET                 DS       CL30
03C                             ORG      ADDR+30
01E      NUMBER                 DS       CL4
022      STR                    DS       CL26
03C      CITY                   DS       CL20
050                             ORG      STR+26
03C      PCODE                  DS       CL4
040      CITYN                  DS       CL16
                                ORG      ADDR+80
         $IS$                   CSECT

         ADDRESS                DSECT
000      ADDR                   DS       CL80
050                             ORG      ADDR
000      NAME                   DS       CL30
01E      STREET                 DS       CL30
03C      CITY                   DS       CL20
050                             ORG      ADDR
000      LNAME                  DS       CL14
00E                             DS       CL2
010      FNAME                  DS       CL14
01E      NUMBER                 DS       CL4
022      STR                    DS       CL26
.        .                      .        .
.        .                      .        .
```

EQU - Equates

Special Character Equates			
#CR	Carriage Return	X'15'	
#SP	Blank	X'40'	C' '
#DOT	Dot	X'4B'	C'.'
#EOM	End of message	X'4E'	C'+'
#END	Enditem (!,#)	X'4F'	C'!'
#FC	Asterisk	X'5C'	C'*'
#HYPH	Dash	X'60'	C'-'
#SLASH	Slash	X'61'	C'/'
#SOM	Start of message	X'6E'	C'>'
#CHA	At-sign	X'7C'	C'@'
#APO	Apostrophe	X'7D'	C'''

Equates are used to attach a value to a label. They are very often used for length calculations.

Examples:

```
        VALUE1  EQU  20              ⇨ X'14' / 20₁₀
```

```
        FIELDA  DS   CL40
        VALUE2  EQU  L'FIELDA        ⇨ X'28' / 40₁₀
```

```
01E     NBR     DS   CL4
022     STREET  DS   CL26
03C     PCODE   DS   CL4
040     VALUE3  EQU  *               ⇨ X'40' / 64₁₀
```

```
        VALUE4  EQU  PCODE-NBR       ⇨ 3C₁₆-1E₁₆ = 1E₁₆
```

```
        VALUE5  EQU  STREET+5        ⇨ 022₁₆+5₁₆ = 27₁₆
```

```
        VALUE6  EQU  X'44'           ⇨ X'44' / 68₁₀
```

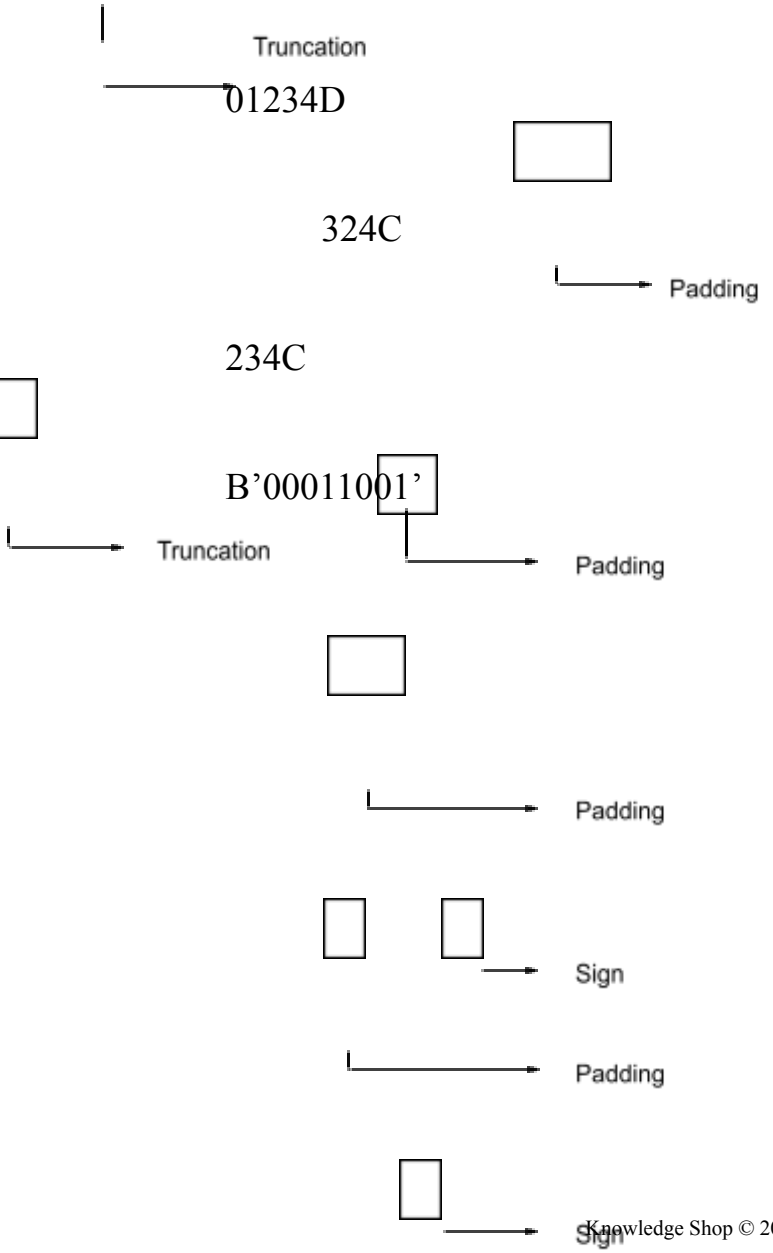
```
        VALUE7  EQU  C'A'            ⇨ X'C1'
```

```
        VALUE8  EQU  B'01111111'     ⇨ X'7F' / 127₁₀
```

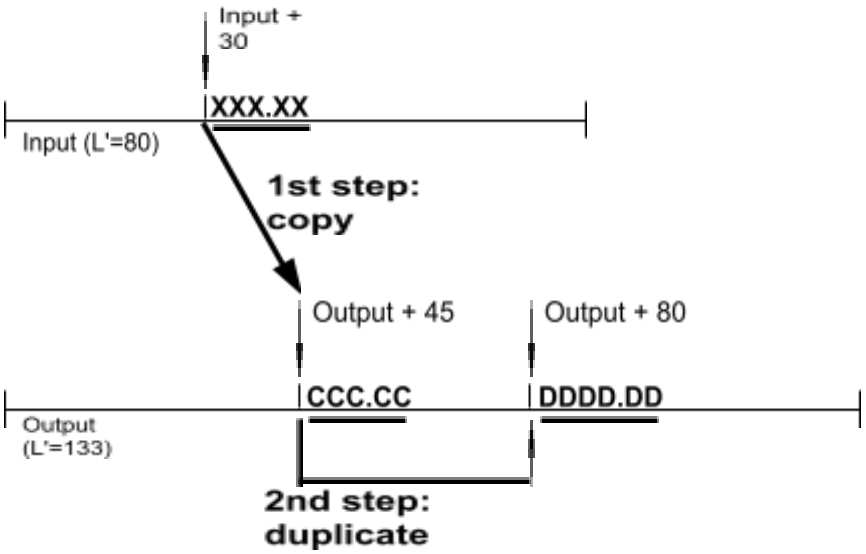
Padding / Truncation

Padding = fill up / complete
Truncation = cut off

```
FIELDA   DC   CL4'ABCDEF'        C1C2C3C4

FIELDB   DC   CL6'1234'          F1F2F3F44040

FIELDC   DC   XL2'89ABCDE'       BCDE

FIELDD   DC   XL4'567AB'               000567AB

FIELDE   DC   P'-1234'           01234D

FIELDF   DC   P'324'                   324C

FIELDG   DC   PL2'1234'          234C

FIELDG   DC   B'11001'           B'00011001'
```



Desk assembly



Input XXX.XX 3 decimal digits followed by a dot and then again 2 decimal digits
i.e. 123.45 → EBCDIC: F1 F2 F3 4B F4 F5
075.01 → EBCDIC: F0 F7 F5 4B F0 F1
999.99 → EBCDIC: F9 F9 F9 4B F9 F9

Task The input number is to be copied from the input line to the output line (1st step).
As next the copied number is to be duplicated and the result is also placed on the output line (2nd step).

Notes Duplication is to be executed using binary arithmetics, i.e. the dot (X'4B') is to be removed for proper conversion and calculation.
The same procedure us used the way back to put the result on the output line, i.e. the dot is to be inserted at the correct location.
DSECTs / fields and base registers see on next page.
The binary value should be placed within register R5 for calculation. To duplicate the value use the Assembler instruction AR (Add register): i.e. AR R5,R5.

Steps 1. coding (after reviewing some solutions hand out of the course' coding solution
2. fill in the location counter (review of your solutions, hand out of 'our' solution)
3. translation of the source code into the machine code (review, hand out of the final solution)

Desk assembly: coding

(Solution will be handed out; solution in ASSDesk.doc)

Location counter	Machine code	**Code (editor in TSO)**
000	8 bytes of macro coding	BEGIN
	64 bytes of machine coding	some Assembler instructions
...		**MVC** ...
	152 bytes of machine code	some more Assembler instructions
		FINIS

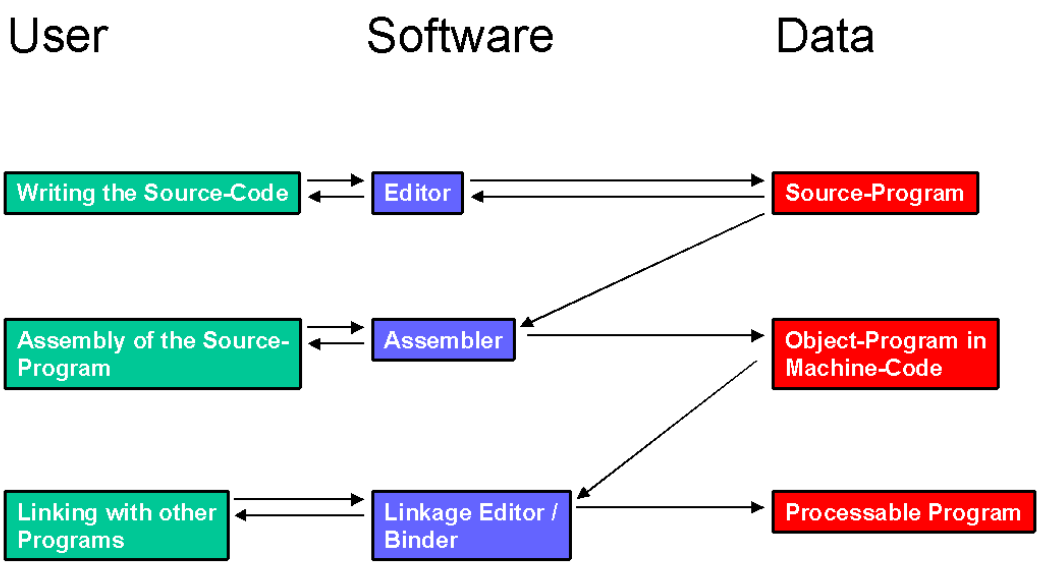
DSECTs / field definitions

```
            WORK        DSECT
000         268 bytes of previous definitions
...         CONVERT     DS       D
            TASK1       DSECT
...         INPUT       DS       CL80
...         OUTPUT      DS       CL133
            $IS$        CSECT
                        USING  WORK,R7
                        USING  TASK1,R3
```

Assembler instructions

A short introduction to the Assembler

It is not possible to communicate directly with a machine using a human language. Therefore we have to establish a code by which we can pass information to the machine or computer. The nature of this code is determined by the requirements and capabilities of the machine in question. If we use electric current, which creates tiny magnetic fields, the machine can distinguish between two states: positive and negative. If the magnetic fields (states) are lined up sequentially, we can define particular combinations of states to equate to a human alphabet. Thus it is possible to code our language in such a way that a machine can process it.
The above kind of code is called machine code, and is based on the binary system. Thus each of "our" characters is defined as a particular combination of "ones" and "zeroes". The definitions are internationally standardised in code-tables. Large systems often use the EBCDIC code-table (Extended Binary-Coded Decimal Interchange Code): here "A" = "11000001". Personal Computers almost exclusively use the ASCII code-table (American Standard Code for Information Interchange): here "A" = "1000001".
In the early days of data processing a programmer had to write his program in the machine code format, a sequence of states (bits) like this: 1101001000000001... . A little help here is the hexadecimal display of the machine code: D2015014F02E. But as you can see this 1st generation language is hard to understand and to learn. The hexadecimal string shown above is one instruction! Imagine the work involved and the possibilities of errors in a program with hundreds or thousands of instructions written in this machine language. To simplify the programmer's work, other programming languages were developed, which use mnemonic operation codes and symbolic addresses. The production or the maintenance of programs is much easier now. These alphanumeric symbols are more declarative and easier to understand than a hexadecimal string.
The assembler language is the next-higher programming language of the machine language. It's a machine-orientated language of the 2nd generation, using symbolic instructions. 2nd-generation means that the conversion of one assembler instruction results in one machine-code instruction. (Example: MVC FIELDA,FIELDB == D2015014F02E).
Once again, each instruction in the source code is assembled to a single instruction in machine language. This conversion is done by a translation program, a compiler (here called assembler). After the conversion, there are two programs: one program still in source code, and one program in machine code. The other translation program is called an interpreter. This program also translates source code to machine code, but in a different way: after translation of a single 'source' instruction, the resulting 'machine' instruction will be executed immediately. In the end, there is no program in machine code. If the program starts again, it has to be translated (interpreted) again. Certainly a compiler-program is more complicated, but in the end it's faster and cheaper than an interpreter-program.
The machine orientation and the lack of structured programming are reasons an assembler programmer uses or needs more time to code a program. But the product has the advantage of using the capabilities of the computer in a more direct and efficient way.
The coding of a program in a language of the 3rd or a higher generation, for example COBOL, PASCAL, C or PL/1, is often simpler and faster. These languages are problem-orientated and their structures and symbolic conventions are closer to the human language. Therefore they are easier to learn.
There are, then, three levels of computer languages: machine language, symbolic language (assembler), and high-level language. The computer only understands the bits comprising the machine language instruction.



A Add fullword

Mnemonic | Operands

A	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

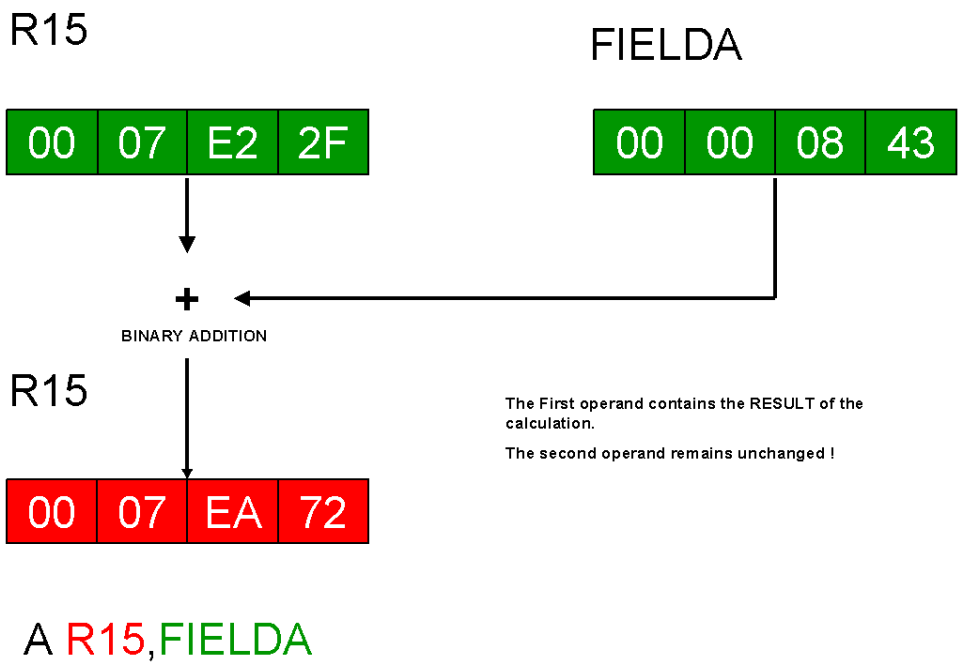
5 A	R_1	X_2	B_2	D_2
0	8	12	16	20 31

A four byte field in storage containing a binary value has to be added to the binary value of a register. The A (Add fullword) instruction handles this problem. The data of the 2nd-operand field, a fullword (->> boundary!) in storage, is added to the contents of the 1st-operand register and the resulting sum is placed in the 1st-operand register. The operands and the result are treated as 32-bit signed binary integers. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.
The 2nd field, a fullword (->> boundary!) in storage, is left unchanged.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

Graphical example



AH Add Halfword

Mnemonic	Operands
AH	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

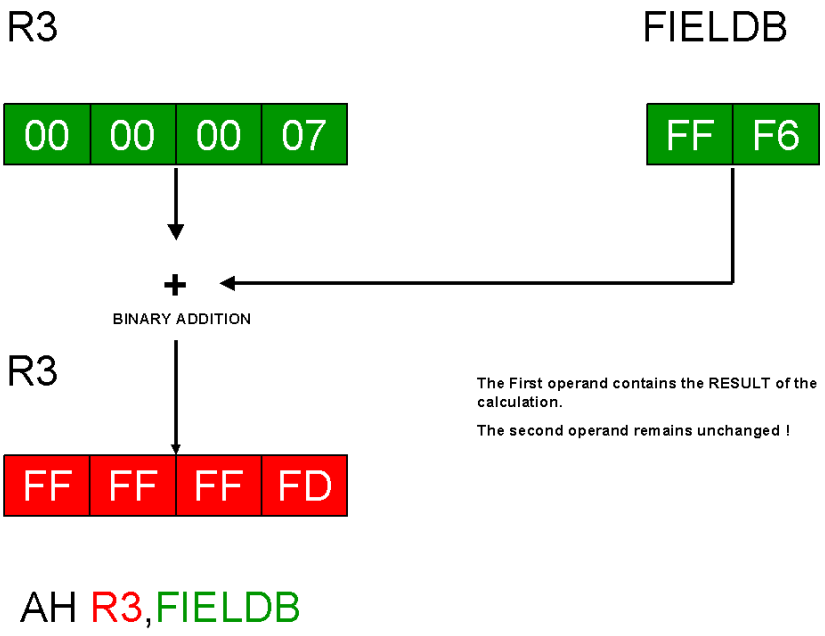
4 A	R_1	X_2	B_2	D_2
0	8	12	16	20 31

A two byte field in storage containing a binary value has to be added to the binary value of a register. The problem can be solved by using the AH (Add Halfword) instruction. The data of the 2nd-operand field, a halfword (->> boundary!) in storage, is added to the contents of the 1st-operand register and the resulting sum is placed in the 1st-operand register. The 2nd-operand field is treated as a 16-bit signed binary integer. The 1st-operand register and the result are treated as 32-bit signed binary integers. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

Graphical example



AP Add packed decimal data

Mnemonic | Operands

AP	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

Machine format (SS2)

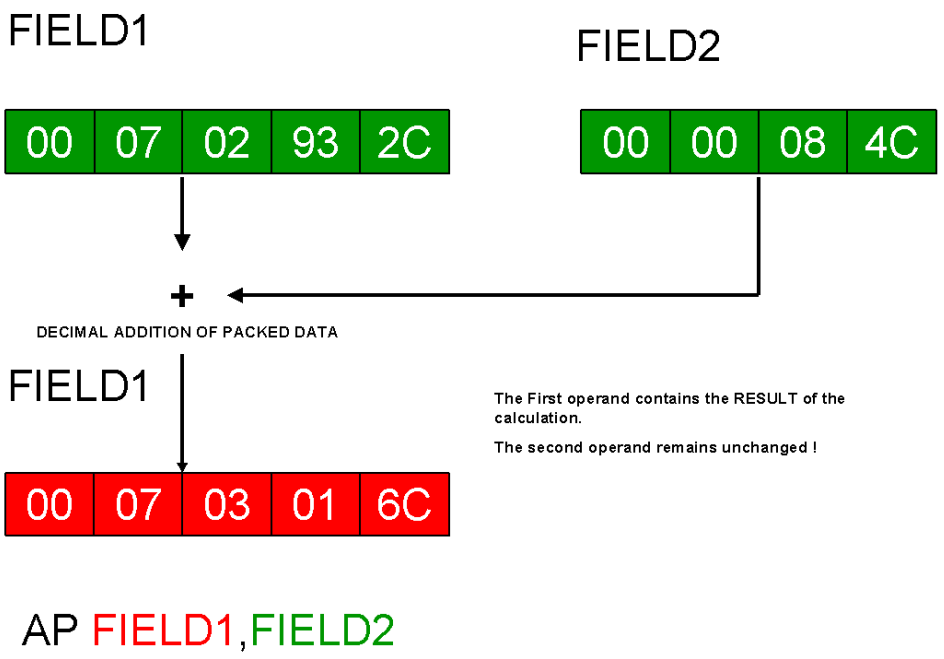
F A	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

Two fields containing packed decimal data have to be added. The instruction AP (Add Packed decimal data) will do this in one step. The contents of the 2nd-operand field are added to the contents of the 1st-operand field and the resulting sum is placed in the 1st-operand field. The initial contents of both operand fields have to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The result is also in packed decimal format. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. In case of an overflow the overflow digits are ignored and additionally the condition code is set to 3.a condition code

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand field

Graphical example



AR Add register

Mnemonic | Operands

AR	R_1 , R_2

Machine format (RR)

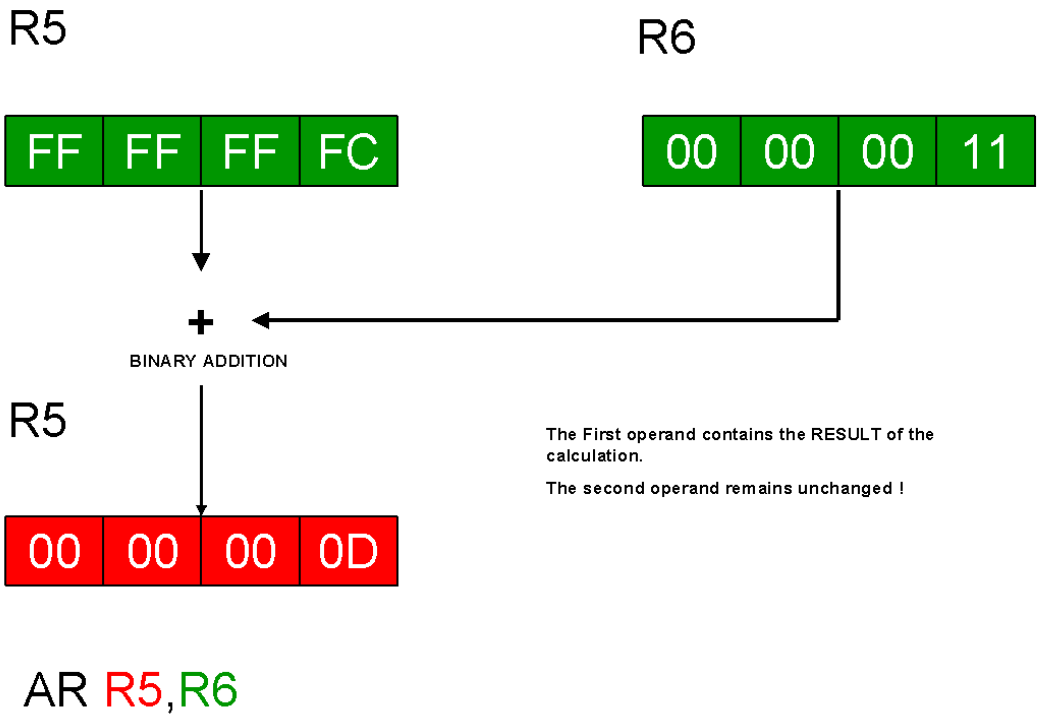
1 A	R_1	R_2
0	8	12 15

To add the contents of two registers, the AR (Add Register) instruction is performed. The contents of the 2nd-operand register are added to the contents of the 1st-operand register and the resulting sum is placed in the 1st-operand register. The operands and the result are treated as 32-bit signed binary integers. The 2nd-operand register remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.rmed. The contents of the 2nd-operand remain unchanged.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

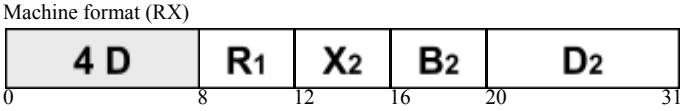
Graphical example



BAS Branch and save

Mnemonic	Operands
BAS	R_1 , D_2 (X_2 , B_2)

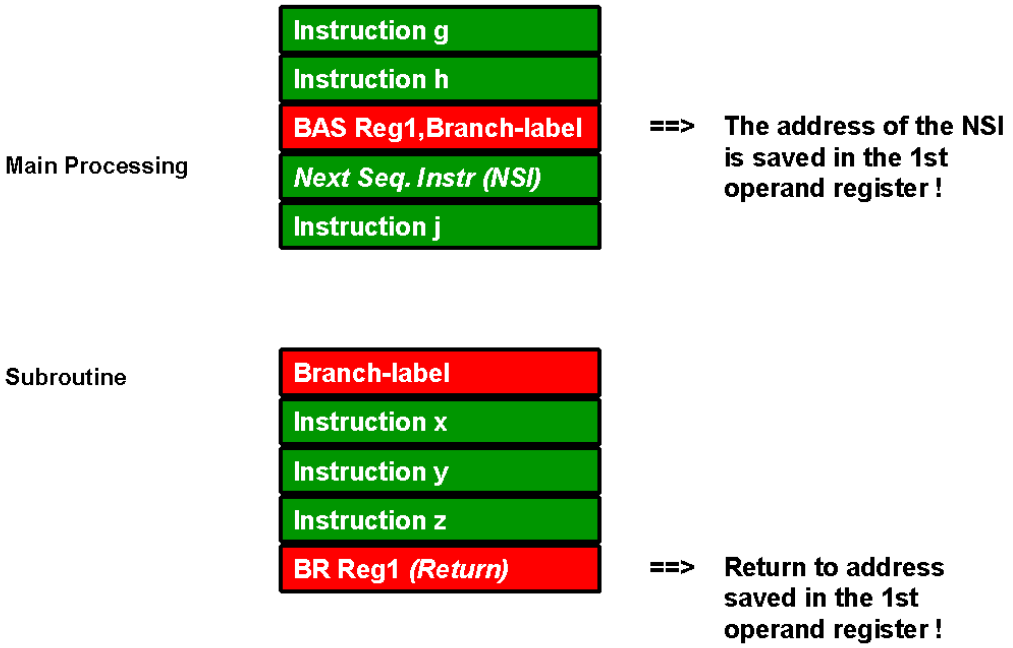
Machine format (RX)



Unconditional branching within a program and the possibility to return to the branch-point is performed by the BAS (Branch And Save) instruction. The address of the next sequential instruction (NSI) is placed in the 1st-operand register. Subsequently a branch is made to the location given by a 2nd-operand label.
The instruction(s) is commonly used to branch to a subroutine with an expected return. Now to return, the help of the unconditional BR (Branch Register) instruction is needed (see the BC instruction).
See also the BASR instruction.

Condition codes
Condition code remains unchanged!

Graphical example



BASR Branch and save register

Mnemonic | Operands

BASR	R_1 , R_2

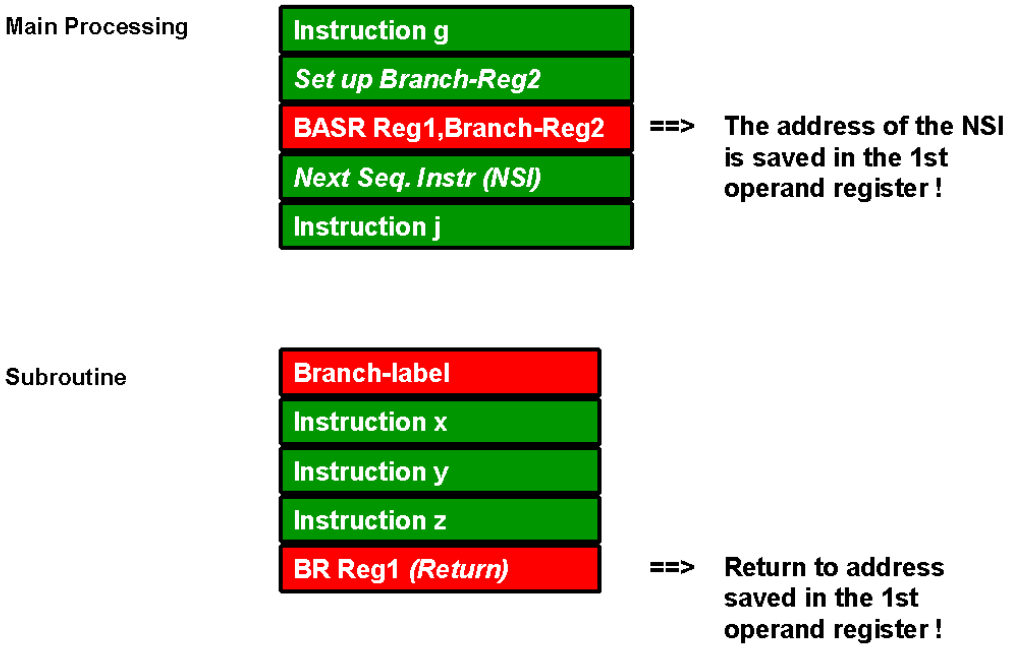
Machine format (RR)

0 D	R_1	R_2
0	8	12 15

Unconditional branching within a program and the possibility to return to the branch-point is also performed by the BASR (Branch And Save Register) instruction. The address of the next sequential instruction (NSI) is placed in the 1st-operand register. Subsequently a branch is made to the location given by the address within the 2nd-operand register.
The instruction(s) is commonly used to branch to a subroutine with an expected return. Now to return, the help of the unconditional BR (Branch Register) instruction is needed (see the BC instruction).
See also the BAS instruction.

Condition codes
Condition code remains unchanged!

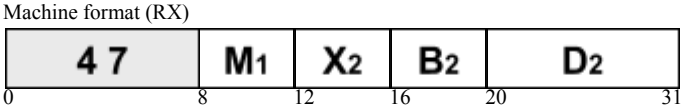
Graphical example



BC Branch on condition

Mnemonic	Operands
BC	M_1 , D_2 (X_2 , B_2)

Machine format (RX)

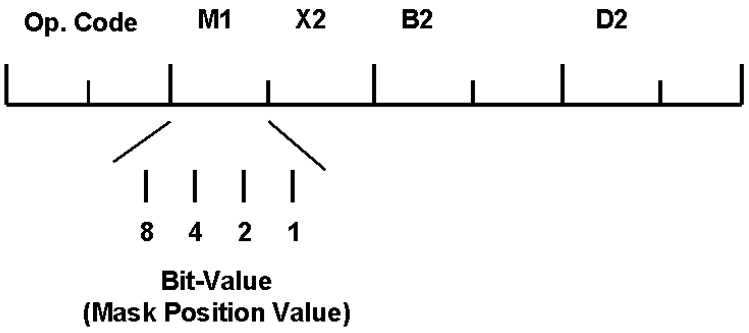


To react on a condition code, which was previously set in the Program Status Word (PSW), the BC (Branch on Condition) instruction is performed. A 4-bit binary mask (1st operand) is used to test the condition code. If the condition is true, a branch is made to the target-location (2nd operand) given by a label; otherwise the next sequential instruction (NSI) is processed.
Instead of coding the 1st-operand mask as in BC 8,LABEL , very often the extended mnemonic instructions are coded as in BZ LABEL . The full set of extended mnemonic branch-instructions is listed in the System/370 Extended Architecture Reference Summary.
See also the BCR instruction.

Condition codes
Condition code remains unchanged!

Graphical example

Condition Code	Mask Position Value
0 (00)	8
1 (01)	4
2 (10)	2
3 (11)	1



BCR Branch on condition register

Mnemonic	Operands
BCR	R_1 , R_2

Machine format (RR)

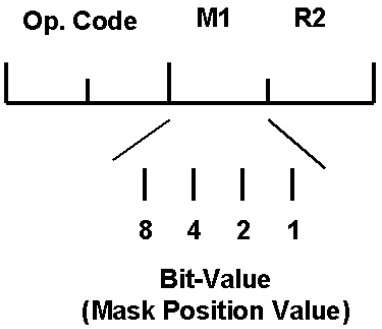
0 7	R_1	R_2
0	8	12 15

To react on a condition code, which was previously set in the Program Status Word (PSW), the BCR (Branch on Condition Register) instruction is performed. A 4-bit binary mask (1st operand) is used to test the condition code. If the condition is true, a branch is made to the target-location (2nd operand) given by the address within a register; otherwise the next sequential instruction (NSI) is processed.
Instead of coding the 1st-operand mask as in
BCR 11,R7 , very often the extended mnemonic instructions are coded as in BNMR R7 . The full set of extended mnemonic branch-instructions is listed in the System/370 Extended Architecture Reference Summary.
See also the BC instruction.

Condition codes
Condition code remains unchanged!

Graphical example

Condition Code	Mask Position Value
0 (00)	8
1 (01)	4
2 (10)	2
3 (11)	1



BCT Branch on count

Mnemonic | Operands

BCT	R_1 , D_2 (X_2 , B_2)

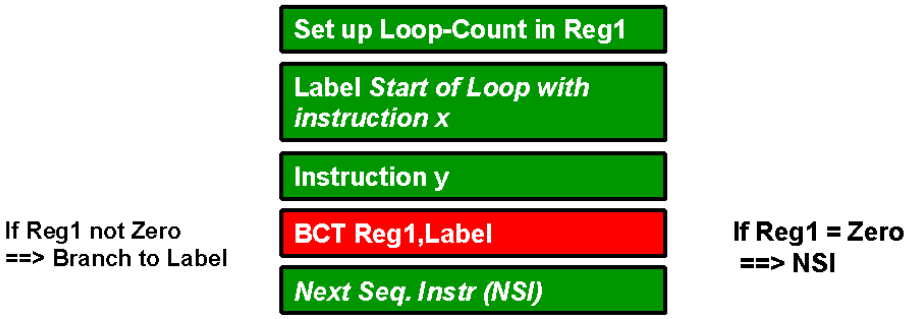
Machine format (RX)

4 6	R_1	X_2	B_2	D_2
0	8	12	16	20 31

One possibility to control a loop is the do times option. The instruction BCT (Branch on CounT) will handle this in following steps. First step: the contents of the 1st-operand register is subtracted by one. Second step: The result is checked for zero. Third step: If the result is not zero a branch is made to the location given by the 2nd-operand label; if the result is zero the next sequential instruction (NSI) is processed.
See also the BCTR instruction.

Condition codes
Condition code remains unchanged!

Graphical example



BCTR Branch on count register

Mnemonic | Operands

BCTR	R_1 , R_2

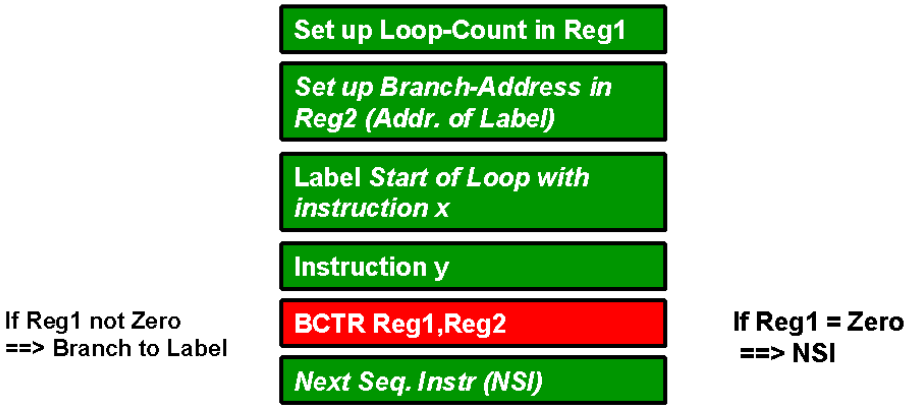
Machine format (RR)

0 6	R_1	R_2
0	8	12 15

One possibility to control a loop is the do times option. Also the instruction BCTR (Branch on CounT Register) will handle this in following steps. First step: the contents of the 1st-operand register is subtracted by one. Second step: The result is checked for zero. Third step: If the result is not zero a branch is made to the location given by the address within the 2nd-operand register; if the result is zero the next sequential instruction (NSI) is processed.
See also the BCT instruction.

Condition codes
Condition code remains unchanged!

Graphical example



BXLE Branch on index low or equal

Mnemonic	Operands
BXLE	R_1 , R_3 , D_2 (B_2)

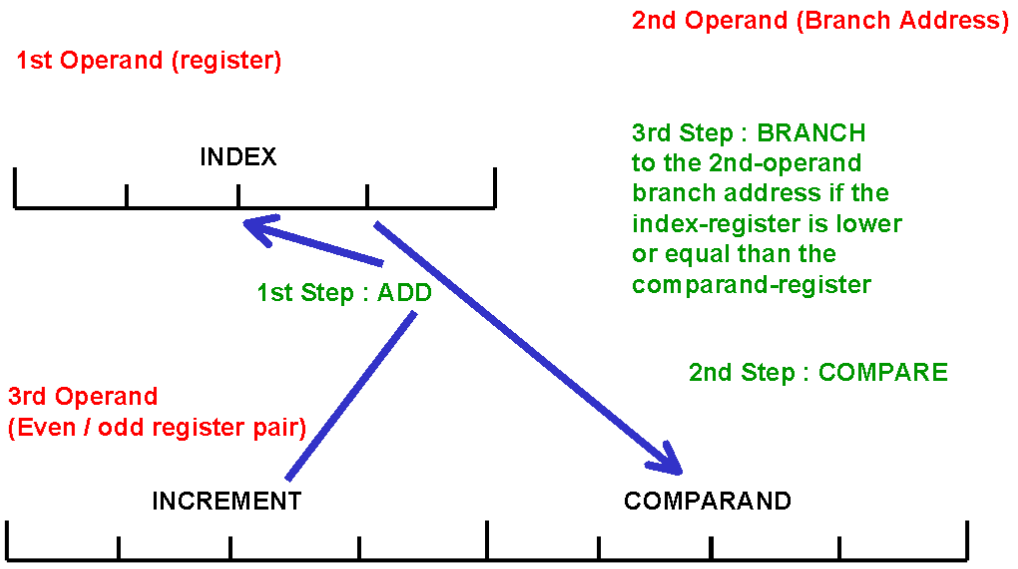
Machine format (RS)

8 7	R_1	R_3	B_2	D_2
0	8	12	16	20 31

One possibility to control a loop is the do until option. The instruction BXLE (Branch on indeX Low or Equal) will handle this in the following steps. First step : An increment is added top the 1st operand index-register. Second step : The result is compared with a comparand. Third step : The result of the comparison determines if a branch occurs or not. If the value of the index-register is low or equal to the value of the comparand a branch is made to the location given by the 2nd operand label; if it is higher the next sequential instruction (NSI) is processed.
The increment and the comparand values are designated by the 2nd operand even-odd register pair.
See also the BXH instruction within the POP on page 7-13 !

Condition codes
Condition code remains unchanged!

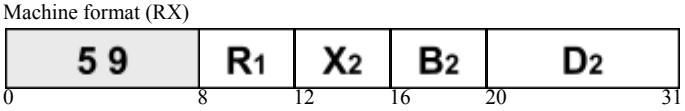
Graphical example



C Compare fullword

Mnemonic	Operands
C	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

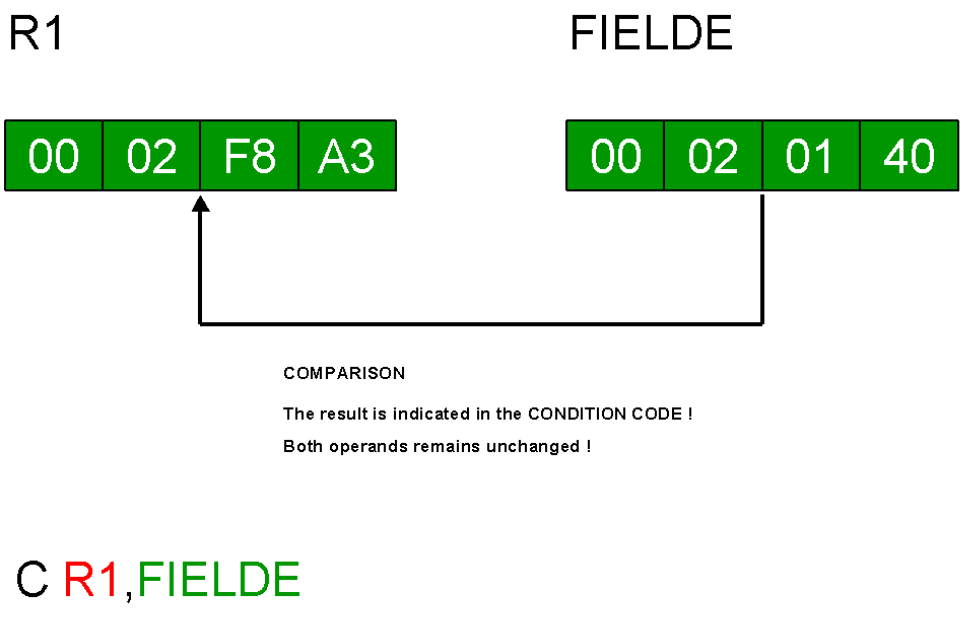


To compare the arithmetic value of a register with the value of a four byte field in storage, the C (Compare fullword) instruction is performed. The data of the 1st-operand register is compared with the data of the 2nd-operand field, a fullword (->> boundary!) in storage. The result of the comparison is indicated in the condition code which may be checked subsequently. The operands are treated as 32-bit signed binary integers and remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand register is lower than the 2nd operand fullword
2	**1. > 2.**	1st operand register is greater than the 2nd operand fullword
3	**not used!**	

Graphical example



C R1,FIELDE

CH Compare Halfword

Mnemonic | Operands

CH	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

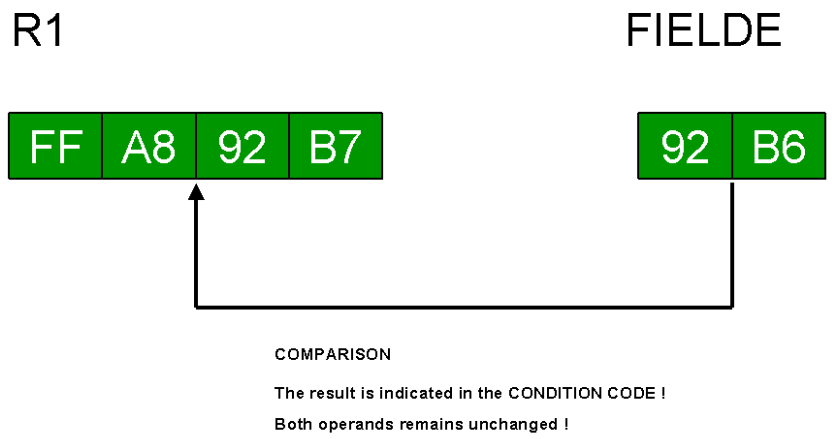
4 9	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To compare the arithmetic value of a register with the value of a two byte field in storage, the CH (Compare Halfword) instruction is performed. It will do this in two steps. First step: the data of the 2nd-operand field, a halfword (->> boundary!) in storage, is extended to a 32-bit interim field. The extension bits are the same as the high-order bit of the 2nd-operand field. Second step: The data of the 1st-operand register is compared with the data of the interim field. The result of the comparison is indicated in the condition code which may be checked subsequently.
The 1st operand is treated as a 32-bit signed binary integer and the 2nd operand is treated as a 16-bit signed binary integer. Both operands remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand register is lower than the 2nd operand Halfword
2	**1. > 2.**	1st operand register is greater than the 2nd operand Halfword
3	**not used!**	

Graphical example



CH R1,FIELDE

CL Compare logical fullword

Mnemonic | Operands

CL	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

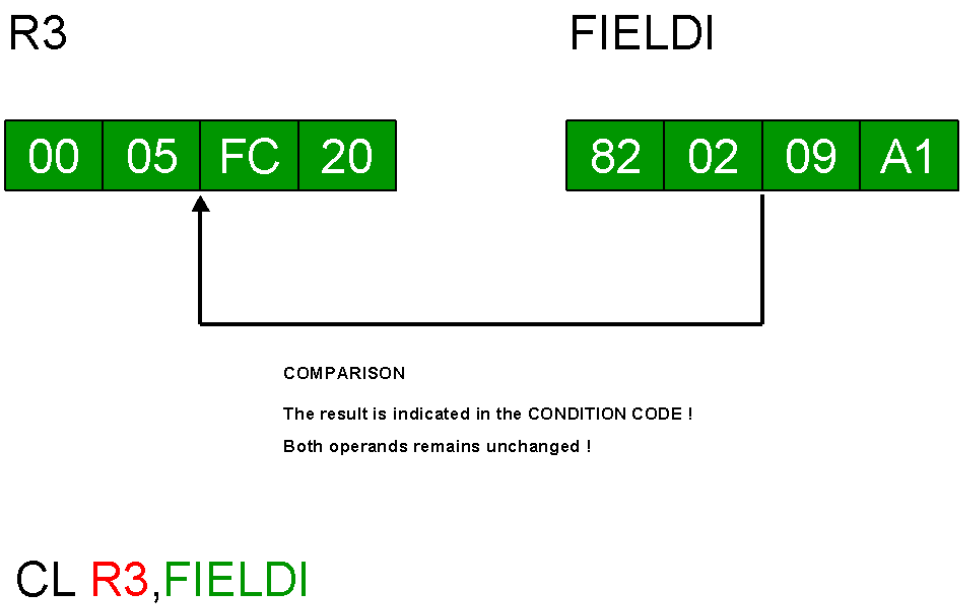
5 5	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To compare logically the contents of a register with the contents of a four byte field in storage, the CL (Compare Logical fullword) instruction is performed. The data of the 1st-operand register is compared with the data of the 2nd-operand field, a fullword (->> boundary!) in storage. Execution proceeds from left to right, bit by bit, and stops as soon as an unequality is recognized or the end of the 1st-operand register is reached. The result is indicated in the condition code which may be checked subsequently.
The sign of the operands does not affect the result of the comparison. Both operands remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand register is lower than the 2nd operand fullword
2	**1. > 2.**	1st operand register is greater than the 2nd operand fullword
3	**not used!**	

Graphical example



CLC Compare logical character

Mnemonic | Operands

CLC	$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

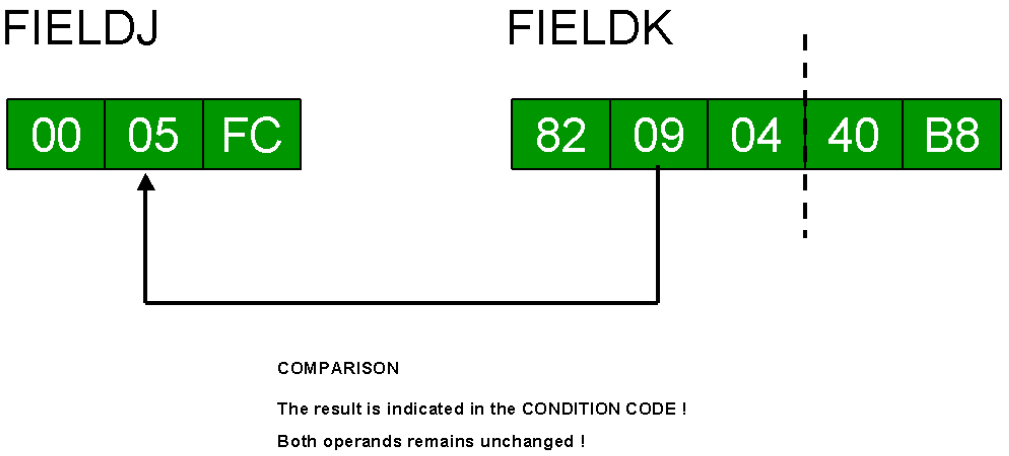
D 5	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

Logical data comparisons of two storage locations are performed by the CLC (Compare Logical Characters) instruction. The data of the 1st-operand field is compared with the data of the 2nd-operand field. Execution proceeds from left to right, bit by bit, and stops as soon as an unequality is recognized or the end of the 1st-operand field is reached. The length is defined by the 1st-operand field (see also the MVC instruction). The result is indicated in the condition code which may be checked subsequently.
The sign of the operands does not affect the result of the comparison. Both operands remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand field is lower than the 2nd operand field
2	**1. > 2.**	1st operand field is greater than the 2nd operand field
3	**not used!**	

Graphical example



CLC FIELDJ,FIELDK

CLCL Compare logical character long

Mnemonic | Operands

CLCL	R_1 , R_2

Machine format (RR)

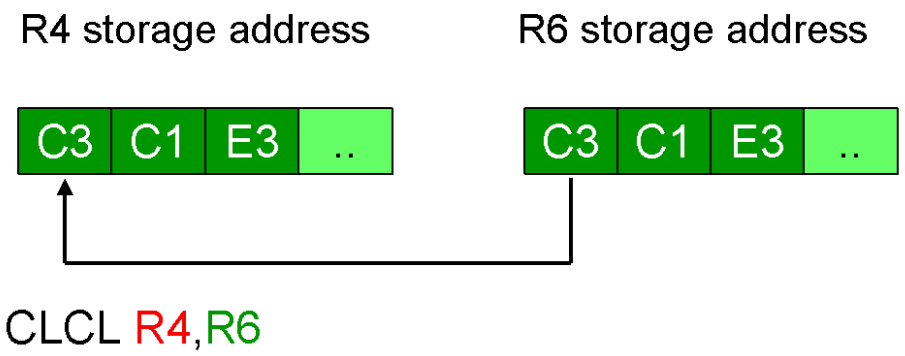
0 F	R_1	R_2
0	8	12 15

Logical data comparisons of two large storage locations are performed by the CLCL (Compare Logical Characters Long) instruction. The data of the 1st-operand field (designated by the 1st-operand register-pair) is compared with the data of the 2nd-operand field (designated by the 2nd-operand register-pair).
The maximum number which can be compared are 16777215 bytes of data. The instruction recommends even-odd pairs of registers. The shorter operand field is considered to be extended on the right with padding bytes. Execution proceeds from left to right, bit by bit, and stops as soon as an inequality is recognized or the end of the larger operand field is reached.
The result is indicated in the condition code which may be checked subsequently. Both operands remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand field is lower than the 2nd operand field
2	**1. > 2.**	1st operand field is greater than the 2nd operand field
3	**not used!**	

Graphical example



The number of bytes to be compared is in R5 (TO-LENGTH) and R7 (FROM-LENGTH) !

The result of the comparison is indicated in the CONDITION CODE !

CLI Compare logical immediate

Mnemonic | Operands

CLI	D_1 (B_1) , I_2

Machine format (SI)

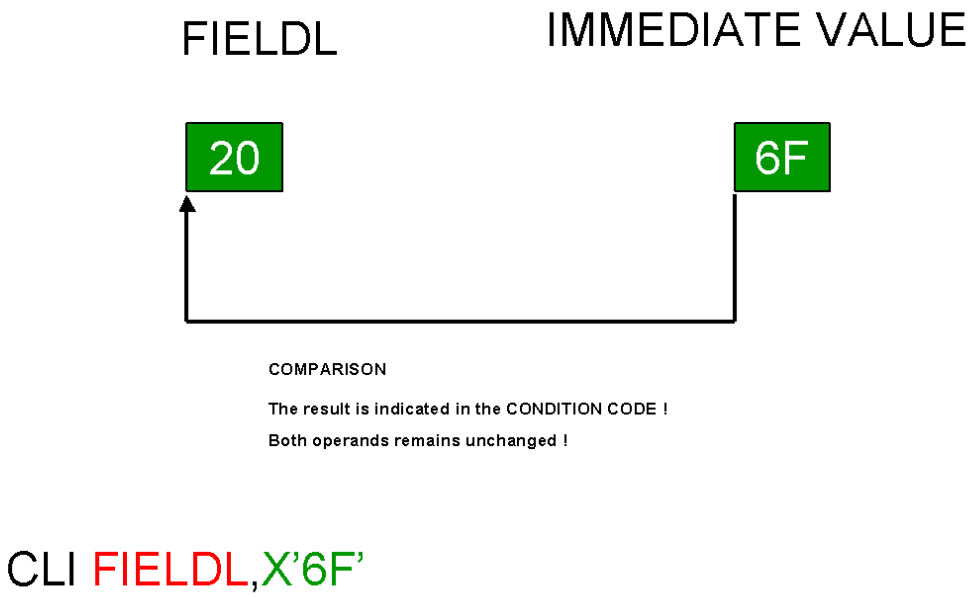
9 5	I_2	B_1	D_1
0	8	16	20 31

To compare logically a single byte of data, defined at program coding, with the data of a storage location, the CLI (Compare Logical Immediate) instruction is performed. The data of the 1st-operand field is compared with the data of the 2nd-operand. The single byte of the 2nd operand is also called the immediate operand (format specifications see the MVI instruction). Execution proceeds from left to right, bit by bit, and stops as soon as an unequality is recognized or the end of the 1st-operand field is reached.
The immediate data to be compared is contained within the instruction itself. The length of the 1st-operand field is not of interest. Only the 1st byte of the field is compared. The result is indicated in the condition code which may be checked subsequently.
The sign of the operands does not affect the result of the comparison. The 1st-operand field remains unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand byte is lower than the 2nd operand immediate value
2	**1. > 2.**	1st operand byte is greater than the 2nd operand immediate value
3	**not used!**	

Graphical example



CLI FIELDL,X'6F'

CLR Compare logical register

Mnemonic | Operands

CLR	R_1 , R_2

Machine format (RR)

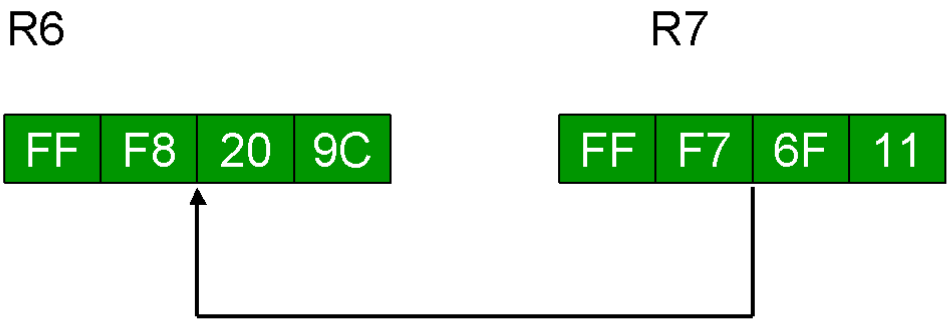
1 5	R_1	R_2
0	8	12 15

Logical data comparisons of two registers are performed by the CLR (Compare Logical Register) instruction. The data of the 1st-operand register is compared with the data of the 2nd-operand register. Execution proceeds from left to right, bit by bit, and stops as soon as an unequality is recognized or the end of the 1st-operand register is reached. The result is indicated in the condition code which may be checked subsequently.
The sign of the operands does not affect the result of the comparison. Both operands remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand register is lower than the 2nd operand register
2	**1. > 2.**	1st operand register is greater than the 2nd operand register
3	**not used!**	

Graphical example



CLR R6,R7

CP Compare packed decimal data

Mnemonic | Operands

CP	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

Machine format (SS2)

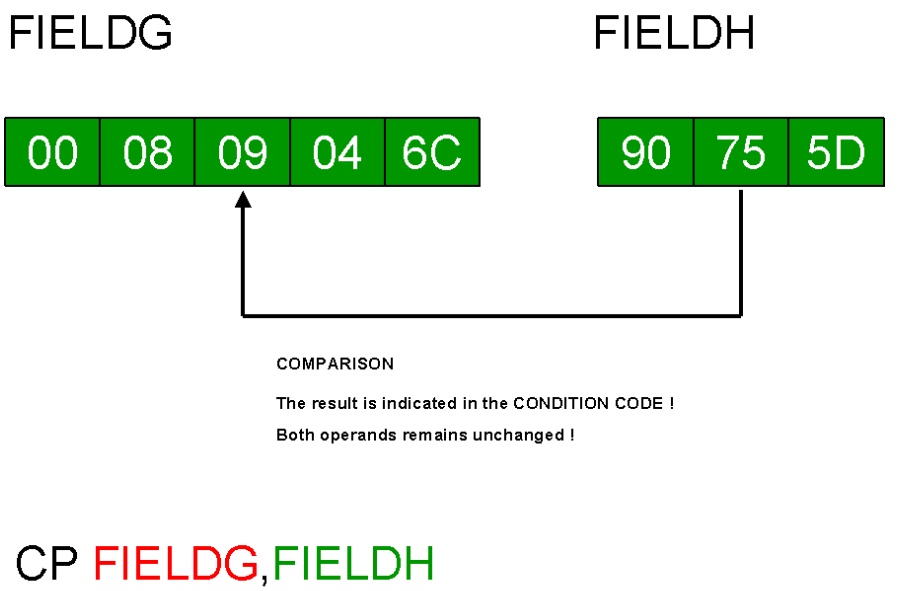
F 9	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

Two fields containing packed decimal data have to be compared. It will be done by the arithmetic compare instruction CP (Compare Packed decimal data). The contents of the 1st-operand field are compared with the contents of the 2nd-operand field. The result of the comparison is indicated in the condition code which may be checked subsequently.
The contents of both fields have to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). Both operands remain unchanged.nary integers and remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand field is lower than the 2nd operand field
2	**1. > 2.**	1st operand field is greater than the 2nd operand field
3	**not used!**	

Graphical example



CP FIELDG,FIELDH

CR Compare register

Mnemonic | Operands

CR	R_1 , R_2

Machine format (RR)

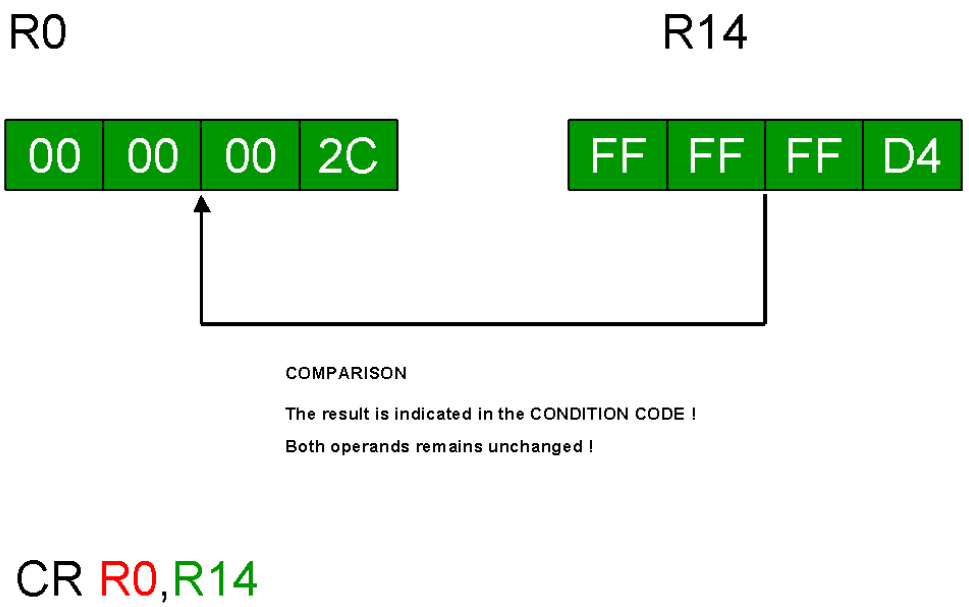
1 9	R_1	R_2
0	8	12 15

To compare the arithmetic values of two registers, the CR (Compare Register) instruction is performed. The data of the 1st-operand register is compared with the data of the 2nd-operand register. The result of the comparison is indicated in the condition code which may be checked subsequently. The operands are treated as 32-bit signed binary integers and remain unchanged.

Condition codes

0	**1. = 2.**	operands are equal
1	**1. < 2.**	1st operand register is lower than the 2nd operand register
2	**1. > 2.**	1st operand register is greater than the 2nd operand register
3	**not used!**	

Graphical example



CVB Convert to binary

Mnemonic | Operands

CVB	R_1 , D_2 (X_2 , B_2)

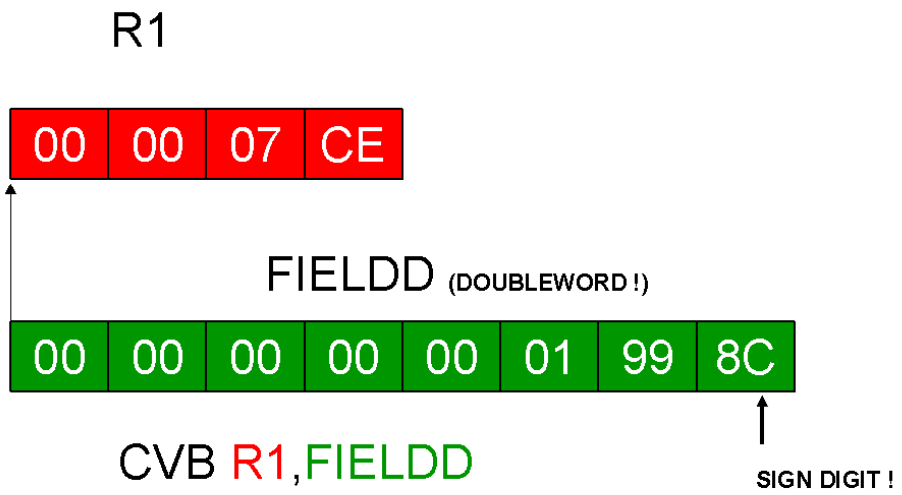
Machine format (RX)

4 F	R_1	X_2	B_2	D_2
0	8	12	16	20 31

The conversion of packed decimal data to binary data is performed by the CVB (ConVert to Binary) instruction. It converts the packed data of the source-field (2nd operand) to binary data and places the result in the target-register (1st operand). The source-field has to be a doubleword (->> boundary!) in storage.
The result is treated as a 32-bit signed binary integer. The maximum positive number that can be converted is 2,147,483,647 (->> X'7FFFFFFF'), the maximum negative number is -2,147,483,648 (->> X'80000000'). If the source-field does not contain packed decimal data, an error occurs (Data Exception). The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



CVD Convert to decimal

Mnemonic | Operands

CVD	R_1 , D_2 (X_2 , B_2)

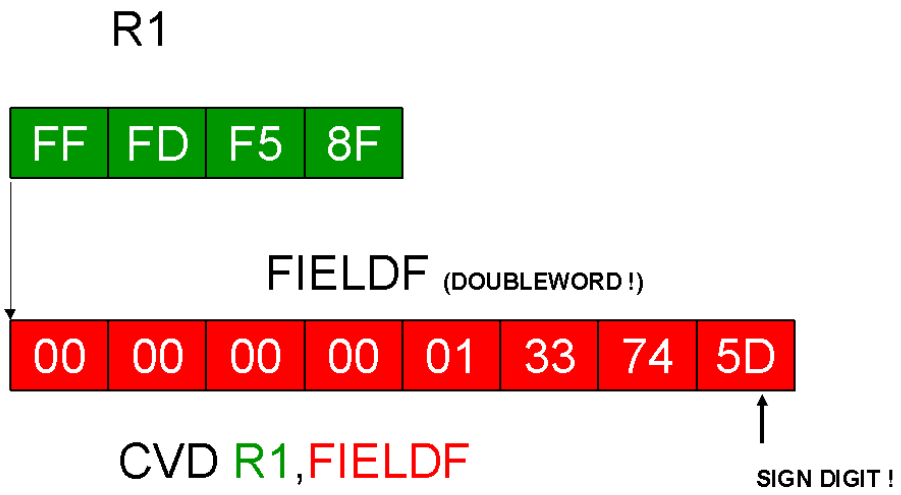
Machine format (RX)

4 E	R_1	X_2	B_2	D_2
0	8	12	16	20 31

The conversion of binary data to packed decimal data is performed by the CVD (ConVert to Decimal) instruction. It converts the binary data of the source-register (1st operand) to packed data and places the result at the target-field (2nd operand). The target-field has to be a doubleword (->> boundary!) in storage.
The source-data is treated as a 32-bit signed binary integer. The result has the format of packed decimal data (coded positive sign-digit is C ; coded negative sign-digit is D). The source-register remains unchanged.

Condition codes
Condition code remains unchanged!

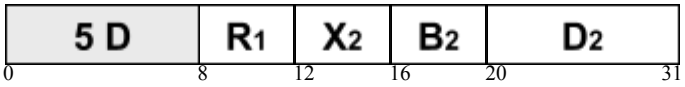
Graphical example



D Divide fullword

Mnemonic	Operands
D	R_1 , D_2 (X_2 , B_2)

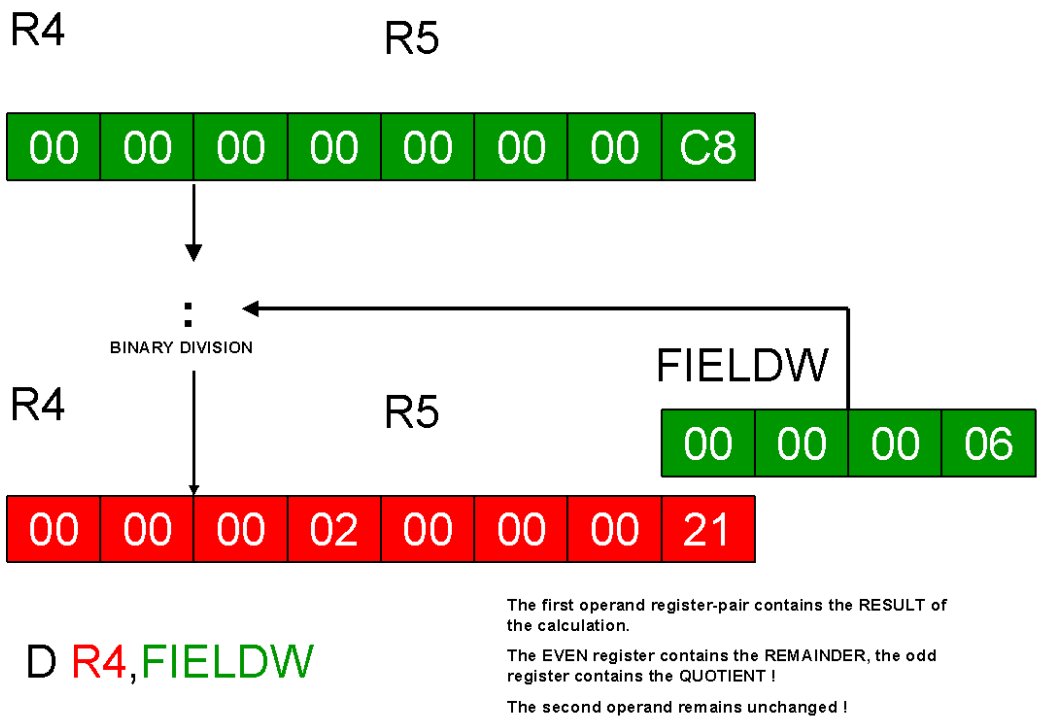
Machine format (RX)



The binary value of a register-pair has to be divided by a four byte field in storage containing also a binary value. The D (Divide fullword) instruction handles this problem. Let's consider the divide instruction first (specially the size of the 1st operand): the problem here is that a division results in a quotient and a remainder. The assembler considers both. The instruction uses an even-odd register-pair to contain the quotient and the remainder. Also the dividend uses this register-pair.
The contents of the 1st-operand register-pair (even-odd) are divided by the data of the 2nd-operand field, a fullword (->> boundary!) in storage. The quotient is placed in the 1st-operand odd-register and the remainder in the even-register.
The 2nd-operand field (divisor), the quotient and the remainder are treated as 32-bit signed binary integers. The 1st-operand register-pair (dividend) is treated as a 64-bit signed binary integer. The remainder takes always the sign of the dividend. The 2nd-operand field remains unchanged. The calculation follows algebraic rules.

Condition codes
Condition code remains unchanged!

Graphical example



DC Define constant

Mnemonic	Operands
DC	Name; Storage type; Length; Value

Assembler Language instructions tell the computer what operations to perform. Assembler Instructions are also used to describe data. The **DC** (Define Constant) instruction is used to define the constants in a program.
We must tell the computer everything about the constant: its length, its type, and, most important, its value. Is it 1 or 256 bytes long ? Is it a numeric or a non-numeric constant ? Is it a binary or a decimal constant ? What is the value of the constant ? The following table shows some examples of the instruction.

Examples

LABEL1	DC	C'/'	L' = 1	**X'61'**	
LABEL2	DC	X'F9AB'	L' = 2	**X'F9AB'**	
LABEL3	DC	H'100'	L' = 2	**X'0064'**	**(Boundary !)**
LABEL4	DC	F'-2'	L' = 4	**X'FFFFFFFE'**	**(Boundary !)**
LABEL5	DC	2CL2'GAG'	L' = 2 (2x !)	**X'C7C1C7C1'**	
LABEL6	DC	CL5'NO'	L' = 5	**X'D5D6404040'**	
LABEL7	DC	FL3'12'	L' = 3	**X'00000C'**	**(No Boundary !)**
LABEL8	DC	3XL2'3A8'	L' = 2 (3x !)	**X'03A803A803A8'**	
LABEL9	DC	3FL2'-3'	L' = 2 (3x !)	**X'FFFDFFFDFFFD'**	**(No Boundary !)**
LABEL10	DC	A(22)	L' = 4	**X'00000016'**	**(Boundary !)**
LABEL11	DC	P'35'	L' = 2	**X'035C'**	
LABEL12	DC	AL2(-1)	L' = 2	**X'FFFF'**	**(No Boundary !)**
LABEL13	DC	2P'-4'	L' = 1 (2x !)	**X'4D4D'**	
LABEL14	DC	BL2'01101'	L' = 2	**X'000D'**	
LABEL15	DC	AL1(C'R',X'40',B'101'	L' = 1 (3x !)	**X'D94005'**	**(No Boundary !)**
LABEL16	DC	CL4'ADE'	L' = 5	**X'C1C4C54040'**	

DP Divide packed decimal data

Mnemonic | Operands

DP	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

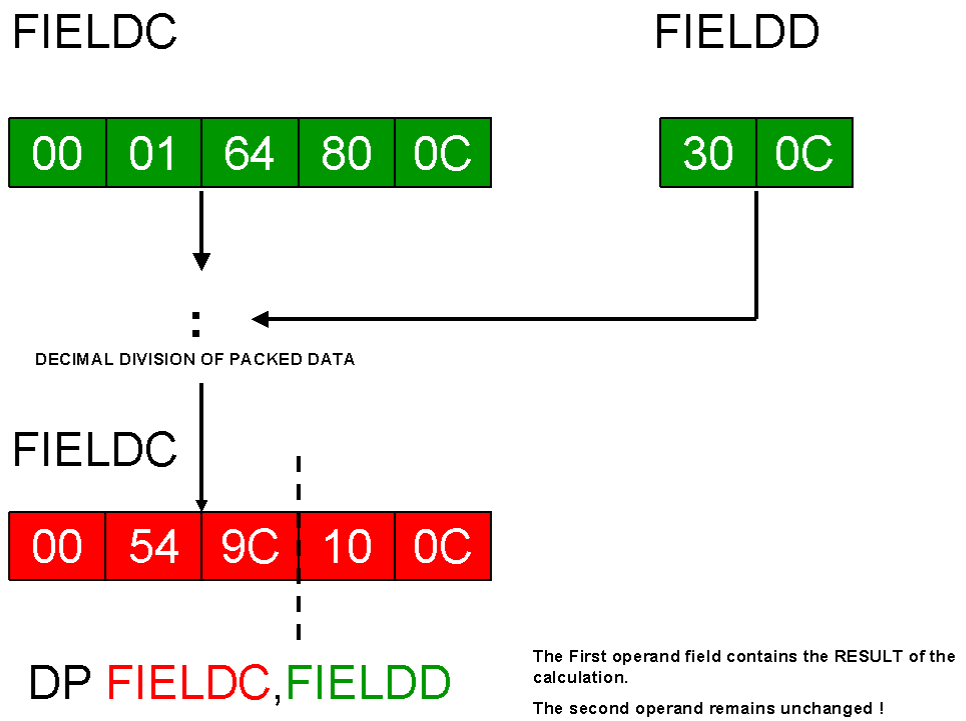
Machine format (SS2)

F D	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

Two fields containing packed decimal data have to be divided by each other. The instruction DP (Divide Packed decimal data) will do this in one step. The contents of the 1st-operand field (dividend) is divided by the contents of the 2nd-operand field (divisor) and the result (quotient and remainder) is placed in the 1st-operand field. The initial contents of both operand fields have to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The results is also in packed decimal format. The 2nd-operand field remains unchanged.
The quotient, in the length of the difference between dividend and divisor (L1-L2!), is placed leftmost in the 1st-operand field. The remainder, in the length of the divisor (L2!), is placed rightmost in the 1st-operand field.
The calculation follows algebraic rules.
Note: The divisor may not be greater than 15 digits plus sign (L2 << 8).

Condition codes
Condition code remains unchanged!

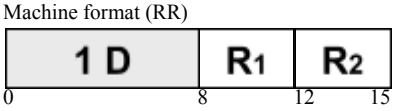
Graphical example



DR Divide register

Mnemonic	Operands
CR	R_1 , R_2

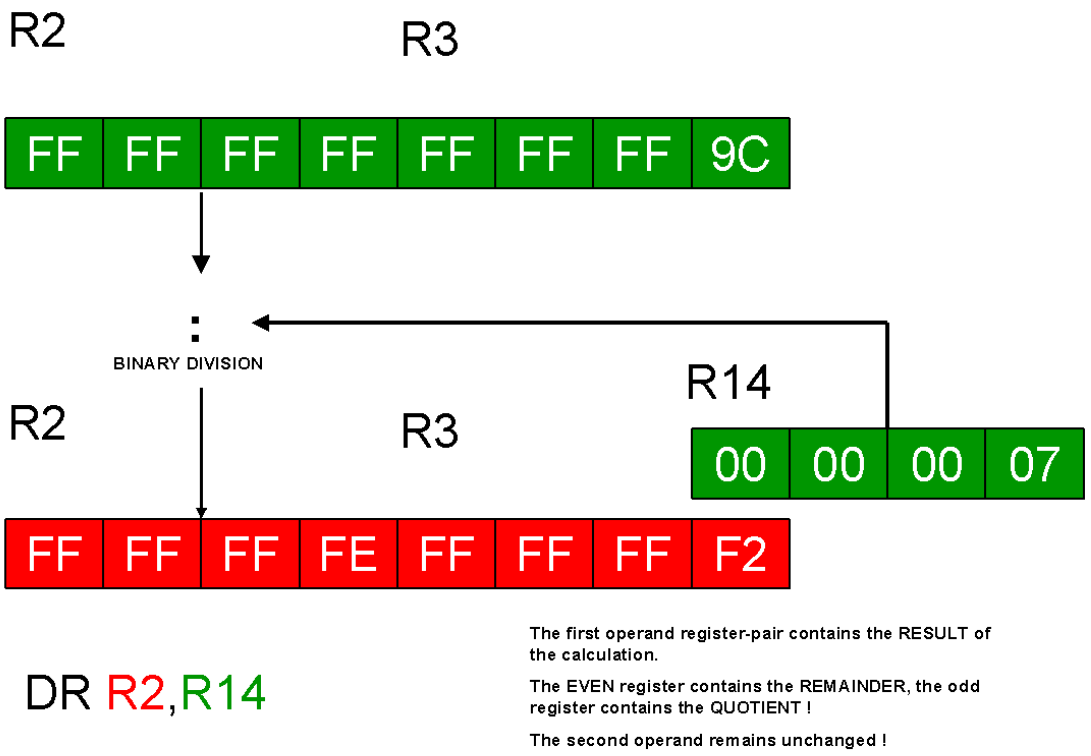
Machine format (RR)



To divide the contents of a register-pair by the contents of another register, the DR (Divide Register) instruction is performed. The considerations about the format of the result and the 1st-operand register-pair are described in the D (Divide fullword) instruction.
The contents of the 1st-operand register-pair (even-odd) are divided by the contents of the 2nd-operand register. The quotient is placed in the 1st-operand odd-register and the remainder in the even-register.
The 2nd-operand register (divisor), the quotient and the remainder are treated as 32-bit signed binary integers. The 1st-operand register-pair (dividend) is treated as a 64-bit signed binary integer. The remainder takes always the sign of the dividend. The 2nd-operand register remains unchanged. The calculation follows algebraic rules.

Condition codes
Condition code remains unchanged!

Graphical example



DS Define storage

Mnemonic	Operands
DS	Name; Storage type; Length;

The **DS** (Define Storage) statement reserves storage without defining a value for that storage area. The following table shows some examples of the instruction.

Examples

LABEL1	**DS**	**C**	Character Variable	**Implicit L' = 1**
LABEL2	**DS**	**X**	Hexadecimal Variable	**Implicit L' = 1**
LABEL3	**DS**	**B**	Binary Variable	**Implicit L' = 1**
LABEL4	**DS**	**P**	Packed Variable	**Implicit L' = 1**
LABEL5	**DS**	**H**	Decimal Variable	**Implicit L' = 2**
LABEL6	**DS**	**F**	Decimal Variable	**Implicit L' = 4**
LABEL7	**DS**	**A**	Address Variable	**Implicit L' = 4**
LABEL8	**DS**	**D**	Decimal Variable	**Implicit L' = 8**
LABEL9	**DS**	**0CL8**	Character Variable, **No Storage Reservation**	**Explicit L' = 8**
LABEL10	**DS**	**FL3**	Decimal Variable, **No Boundary**	**Explicit L' = 3**
LABEL11	**DS**	**3XL6**	Hexadecimal Variable, **3x !**	**Explicit L' = 6**
LABEL12	**DS**	**2HL3**	Decimal Variable, **No Boundary, 2x !**	**Explicit L' = 3**
LABEL13	**DS**	**5A**	Address Variable, **5x !**	**Implicit L' = 4**

ED Edit

Mnemonic | Operands

ED	$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

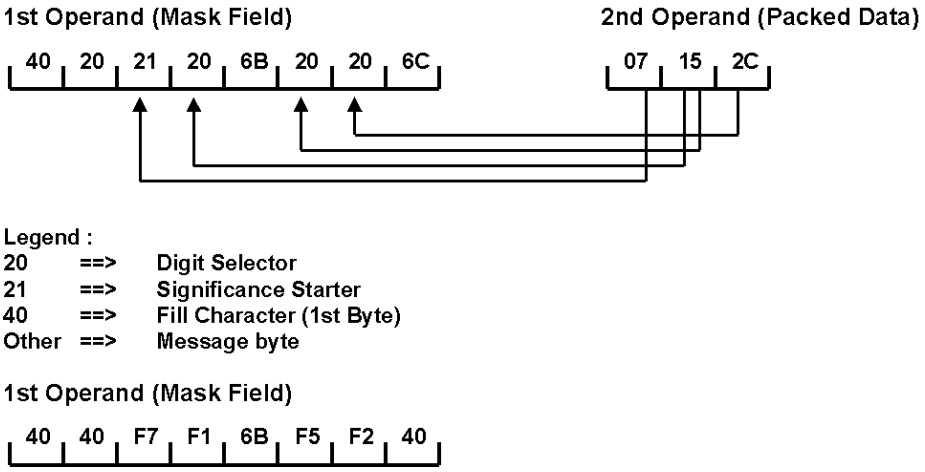
D E	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To edit a field containing packed decimal data to printable characters, the ED (EDit) instruction is performed. The packed data of the 2nd-operand field is changed to the zoned format and placed at right positions in the 1st-operand mask-field, which is also called as the pattern. That mask controls the operation (->> i.e. leading zeros in the 2nd-operand field are 'replaced' by the mask's fill charater). The result replaces the 1st-operand mask-field. The result of the instruction is also indicated in the condition code which may be checked subsequently.
The contents of the 2nd-operand field has to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception).
See also the EDMK instruction.

Condition codes

0	**= 0**	Last Field Zero or Zero Length
1	**< 0**	Last Field less than Zero
2	**> 0**	Last Field greater than Zero
3	**not used!**	

Graphical example




```
ED FIELDA,FIELDB
```

EDMK Edit and mark

Mnemonic | Operands

EDMK	$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

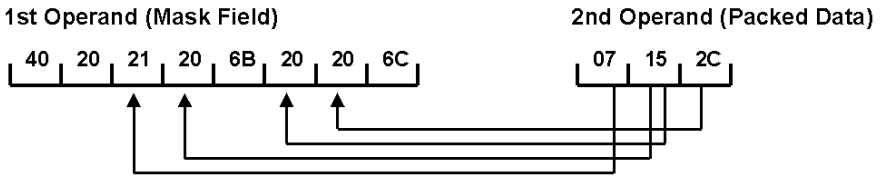
D F	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To edit a field containing packed decimal data to printable characters, the EDMK (EDit and MarK) instruction is performed. The packed data of the 2nd-operand field is changed to the zoned format and placed at right positions in the 1st-operand mask-field, which is also called as the pattern. That mask controls the operation (->> i.e. leading zeros in the 2nd-operand field are 'replaced' by the mask's fill charater). Additionally the address of the 1st significant result byte (1st digit unequal zero) is placed into the register R1. The result replaces the 1st-operand mask-field. The result of the instruction is also indicated in the condition code which may be checked subsequently.
The contents of the 2nd-operand field has to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception).
See also the ED instruction

Condition codes

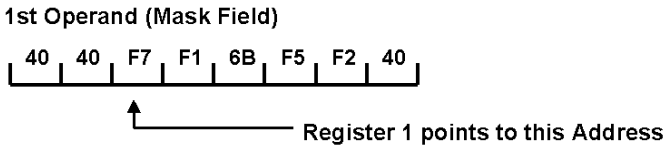
0	**= 0**	Last Field Zero or Zero Lenght
1	**< 0**	Last Field less than Zero
2	**> 0**	Last Field greater than Zero
3	**not used!**	

Graphical example



Legend :

20	==>	Digit Selector
21	==>	Significance Starter
40	==>	Fill Character (1st Byte)
Other	==>	Message byte



EDMK FIELDA,FIELDB

EX Execute

Mnemonic | Operands

EX	R_1 , D_2 (X_2 , B_2)

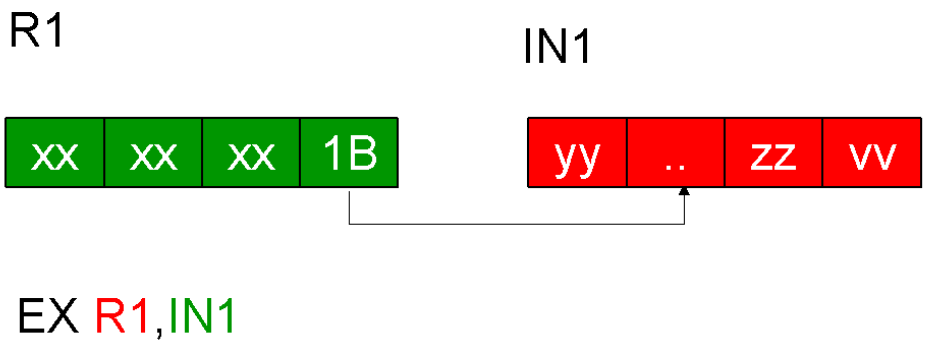
Machine format (RX)

4 4	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To perform SS1 and SS2 instructions with a variable length, the EX (EXecute) instruction is recommended. First the 2nd byte of the instruction at the 2nd-operand address is modified by the rightmost byte of the 1st-operand register (that 2nd byte contains the length attributes!). Afterwards the resulting instruction, called as the target instruction, is executed.
The modification will be done by an OR-connection (see also graphical example).
Be careful using the EX instruction in connection with other instruction formats!
Note: Do NOT use R0 as 1st-operand register !
In this case no OR-connection will be made.

Condition codes
Condition code remains unchanged!

Graphical example



The 2nd byte of the TARGET INSTRUCTION will be 'ORED' with the low order byte contents of the first operand register !

Possible use : MVC with a variable length ...

IC Insert character

Mnemonic | Operands

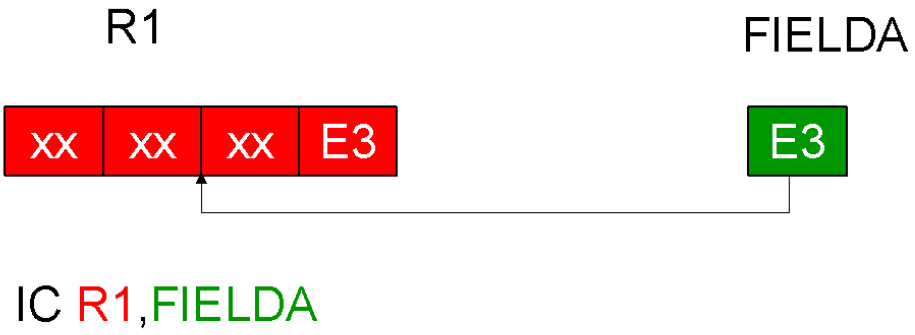
IC	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

4 3	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move one byte with binary data from a storage location into a register, the IC (Insert Character) instruction is performed. The data of the source-field (2nd operand), a single byte in storage, is inserted into the low-order byte of the target-register (1st operand). The source-field and the three high-order bytes of the target-register remain unchanged.

Condition codes
Condition code remains unchanged!



xx - remains UNCHANGED !

Graphical example

ICM Insert character under mask

Mnemonic | Operands

ICM	R_1 , M_3 , D_2 (B_2)

Machine format (RS)

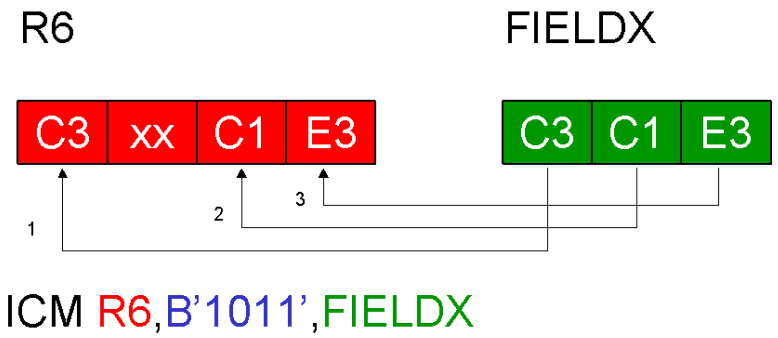
B F	R_1	M_3	B_2	D_2
0	8	12	16	20 31

The ICM (Insert Characters under Mask) instruction is an additional instruction to place one or more bytes of data in storage at wanted positions in a register. The data of the source-field (2nd operand), consecutive bytes in storage, are inserted into the target-register (1st operand) in the positions indicated by the 4-bit mask (3rd operand). The length of the source-field is given by the number of indicated bytes. The source-field and the non-affected bytes of the target-register remain unchanged.
No boundary is wanted! See also the L, LH and IC instructions.
1)* all inserted bits are zeros or mask is zero
2)* leftmost inserted bit is one
3)* leftmost inserted bit is zero, but not all bits are zeros

Condition codes

0 all inserted bits are zero or mask is zero
1 leftmost inserted bit is one
2 leftmost inserted bit is zero, but not all bits are zero
3 **not used!**

Graphical example



Controlled by the 3rd operand MASK : Here = '1011'

L Load fullword

Mnemonic | Operands

L	R_1 , D_2 (X_2 , B_2)

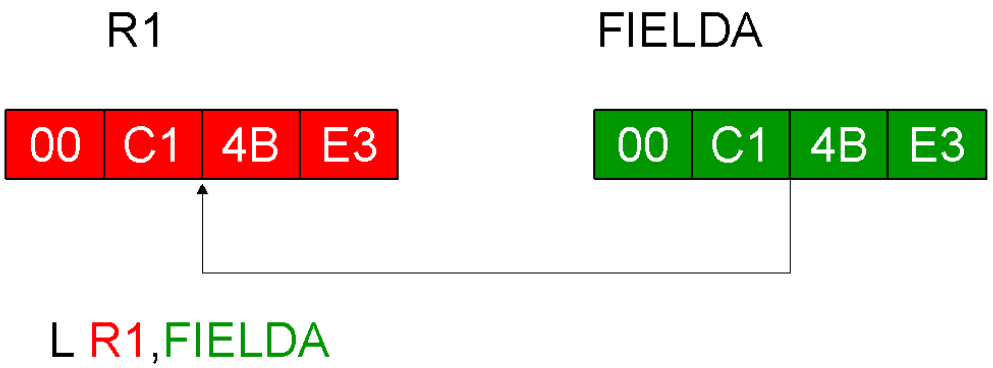
Machine format (RX)

5 8	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move four bytes with binary data from a storage location into a register, the L (Load fullword) instruction is performed. The data of the source-field (2nd operand), a fullword (->> boundary!) in storage, is loaded into the target-register (1st operand). The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



LA Load address

Mnemonic | Operands

LA	R_1 , D_2 (X_2 , B_2)

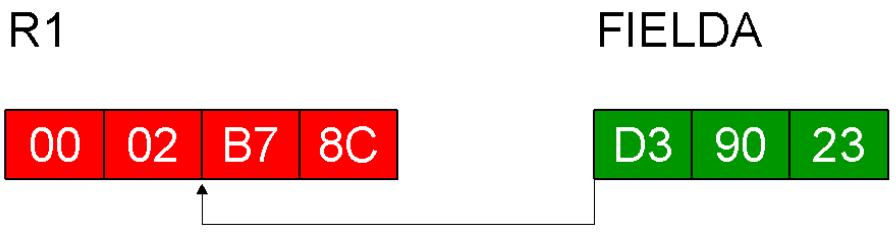
Machine format (RX)

4 1	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To place the address of a storage-location in a register, the LA (Load Address) instruction is performed. The effective address of the 2nd-operand field is loaded in the 1st-operand register. The address is placed right adjusted in the register and if nessesary left padded with zeros.
Please note that the effective address of the 2nd-operand field is calculated adding up the values of the index-register, the base-register and the displacement.
The instruction may also be used to increment the contents of a register. The contents of register 5 are incremented by 8 with the coding: LA R5,8(0,R5)

Condition codes
Condition code remains unchanged!

Graphical example



The ADDRESS is placed in REGISTER 1 !

Calculation - Rule : $X_2 + B_2 + D_2 ==> R_1$

LA R1,FIELDA

LCR Load complement register

Mnemonic	Operands
LCR	R_1 , R_2

Machine format (RR)

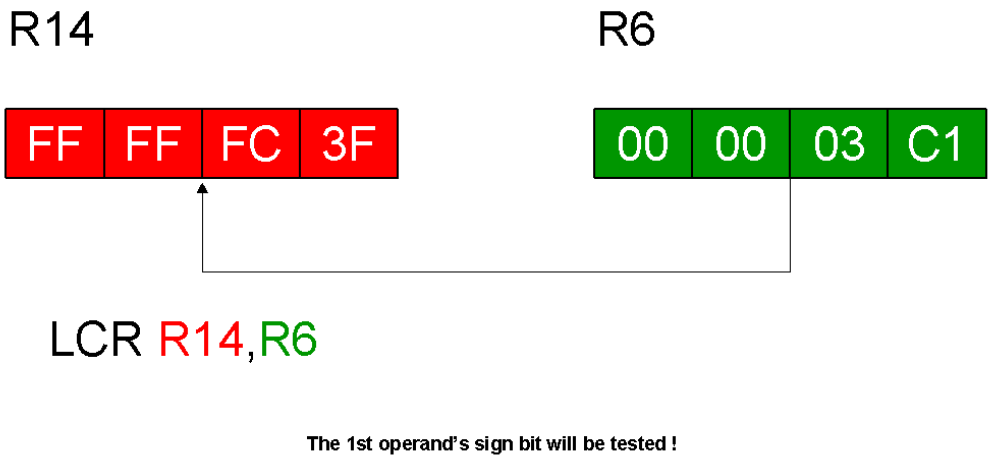
1 3	R_1	R_2
0	8	12 15

To change the binary data of a register to the two's complement of it, the LCR (Load Complement Register) instruction is performed. The contents of the source-register (2nd operand) are changed to the two's complement and the result is loaded into the target-register (1st operand). Afterwards the target-register is tested for sign and magnitude. The result of the test is indicated in the condition code. The source-registers and the result in the target-register are treated as 32-bit signed binary integers. The source-register remains unchanged.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

Graphical example



LH Load Halfword

Mnemonic	Operands
LH	R_1 , D_2 (X_2 , B_2)

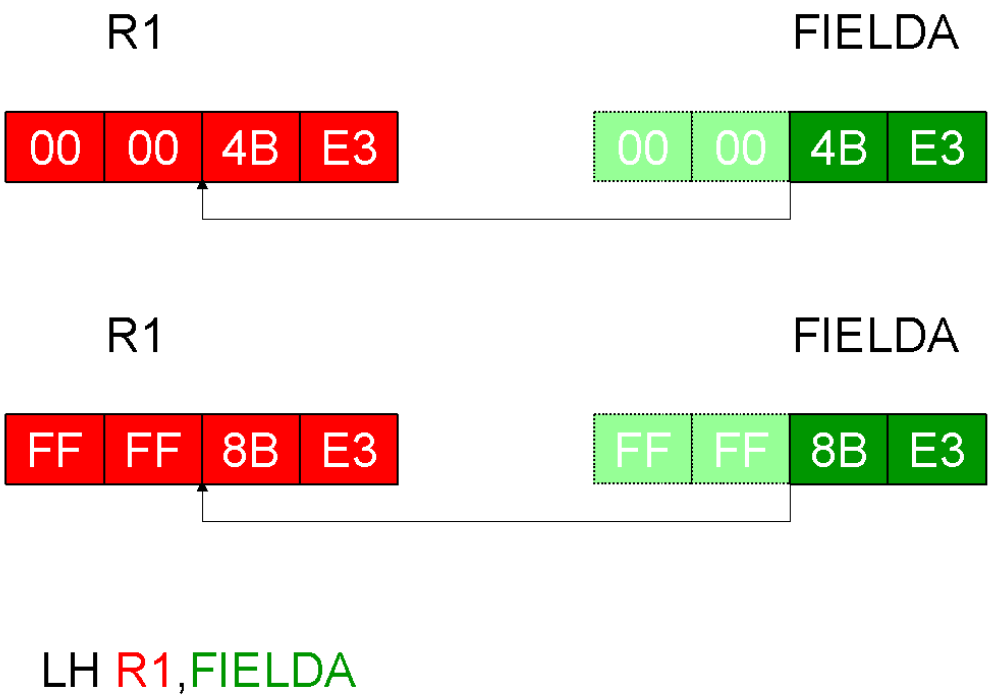
Machine format (RX)

4 8	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move two bytes with binary data from a storage location into a register, the LH (Load Halfword) instruction is performed. It will do this in two steps. First step: the data of the source-field (2nd operand), a halfword (->> boundary!) in storage, is extended to a 32-bit signed binary integer. The extension bits are the same as the high-order bit of the source-field. Second step: the 32 bits are loaded into the target-register (1st operand). The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



LM Load multiple

Mnemonic | Operands

LM	R_1 , R_3 , D_2 (B_2)

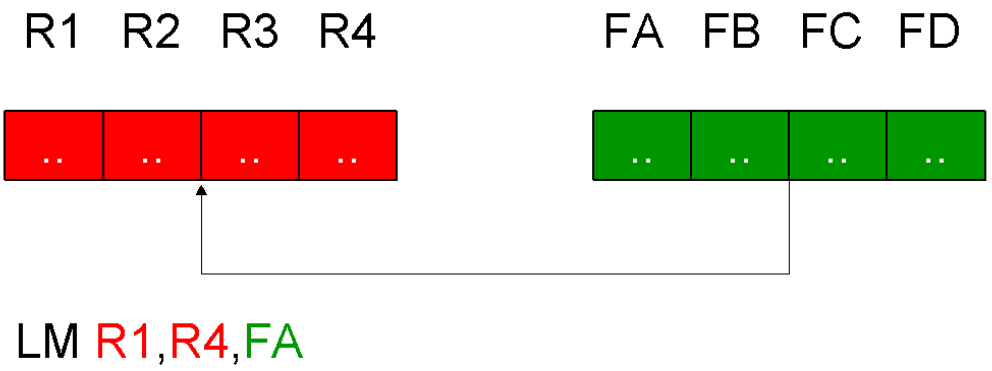
Machine format (RS)

9 8	R_1	R_3	B_2	D_2
0	8	12	16	20 31

To move a sequence of four bytes of binary data from a storage location into adjacent registers, the LM (Load Multiple) instruction is performed. The data of the source-field (2nd operand), a sequence of fullwords (->> boundary!) in storage, is loaded into the adjacent target-registers, which are defined by the starting-register (1st operand) and the ending-register (3rd operand). The target-registers are loaded in the ascending order of their register numbers (with register R0 follows register R15). The number of target-registers defines the length of the source-field. The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example

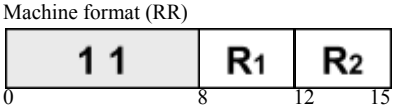


FA, FB, FC and FD are FULLWORDS !

LNR Load negative register

Mnemonic	Operands
LNR	R_1 , R_2

Machine format (RR)

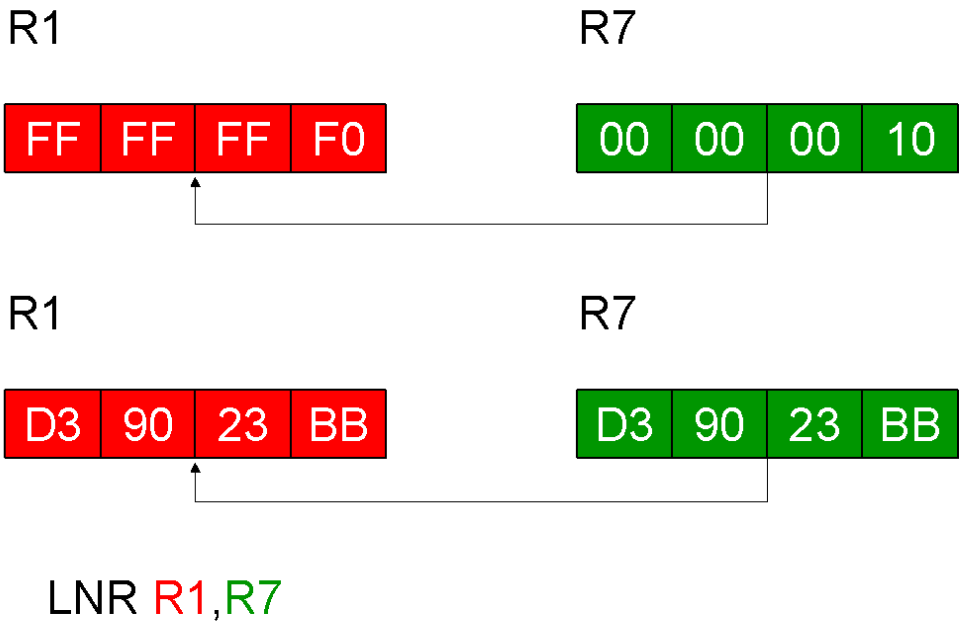


To transfer the two's complement of the absolute value between two registers, the LNR (Load Negative Register) instruction is performed. The binary data of the source-register (2nd operand) are changed into negative and the result is loaded into the target-register (1st operand). If the binary data of the source-register is already negative or zero, the data is tranfered unchanged; otherwise the two's complement is loaded into the target-register. Afterwards the target-register is tested for magnitude. The result of the test is indicated in the condition code. The source-register and the result in the target-register are treated as 32-bit signed binary integers. The source-register remains unchanged.
See also the LPR instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**< 0**	1st operand register is negative
2	**not used!**	
3	**not used!**	

Graphical example



LPR Load positive register

Mnemonic	Operands
LPR	R_1 , R_2

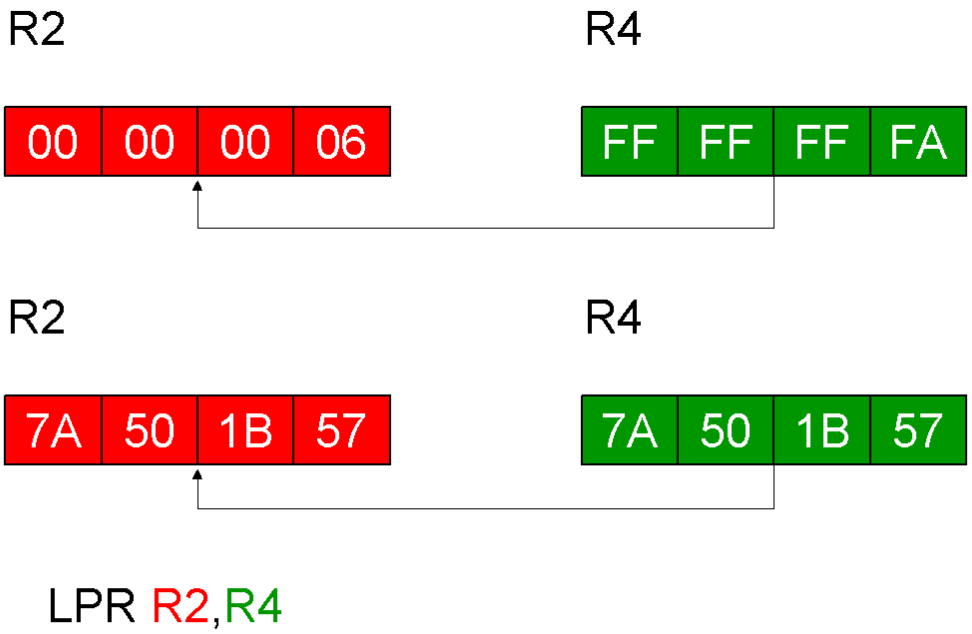
Machine format (RR)

1 0	R_1	R_2
0	8	12 15

To transfer the absolute value between two registers, the LPR (Load Positive Register) instruction is performed. The binary data of the source-register (2nd operand) are changed into positive and the result is loaded into the target-register (1st operand). If the binary data of the source-register is already positive or zero, the data is tranfered unchanged; otherwise the two's complement is loaded into the target-register. Afterwards the target-register is tested for magnitude. The result of the test is indicated in the condition code. The source-register and the result in the target-register are treated as 32-bit signed binary integers. The source-register remains unchanged.

Condition codes

0	**= 0**	1st operand register is zero
1	**not used!**	
2	**> 0**	1st operand register is positive
3	**not used!**	



Graphical example

LR Load register

Mnemonic | Operands

LR	R1 , R2

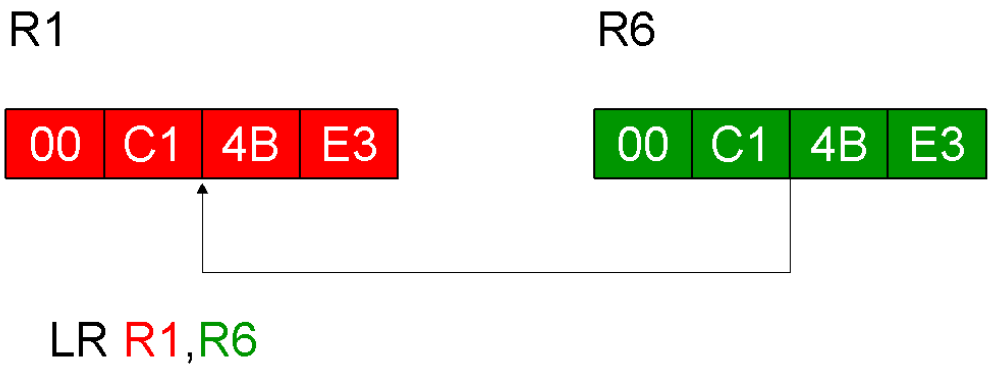
Machine format (RR)

1 8	R1	R2
0	8	12 15

Data transfers between two registers are performed by the LR (Load Register) instruction. The contents of the source-register (2nd operand) are loaded into the target-register (1st operand). The source-register remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



LTR Load and test register

Mnemonic	Operands
LTR	R1 , R2

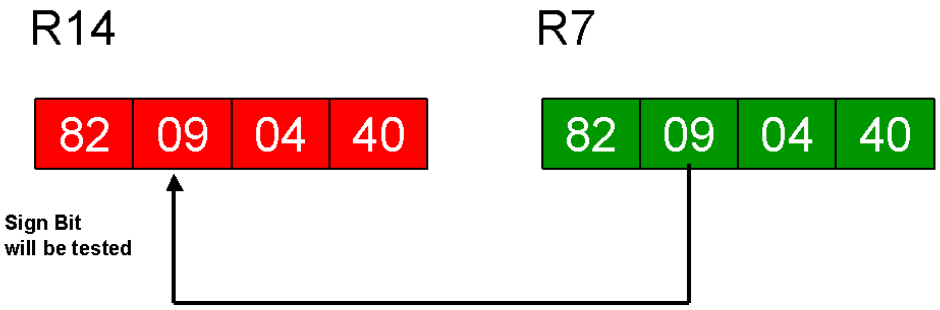
Machine format (RR)

Data transfers between two registers may also performed by the LTR (Load and Test Register) instruction. The contents of the source-register (2nd operand) are loaded into the target-register (1st operand). Additionally the contents of the source-register, treated as a 32-bit signed binary integer, are tested for sign and magnitude. The result of the test is indicated in the condition code. The source-register remains unchanged.
If the two operands are the same register as in LTR R3,R3 , no data is transfered, but the condition code is set!

Condition codes

0	**= 0**	1st operand register is zero
1	**< 0**	1st operand register is negative
2	**> 0**	1st operand register is positive
3	**not used!**	

Graphical example



LTR R14,R7

M Multiply fullword

Mnemonic | Operands

M	R_1 , D_2 (X_2 , B_2)

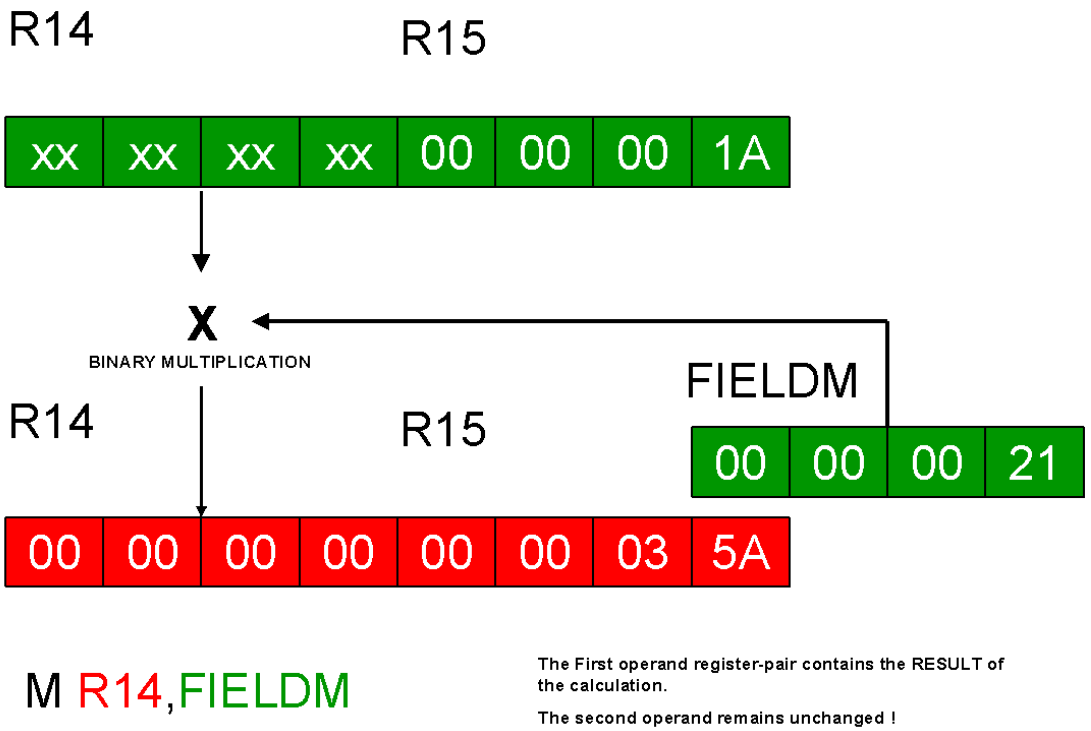
Machine format (RX)

5 C	R_1	X_2	B_2	D_2
0	8	12	16	20 31

The binary value of a register has to be multiplied by a four byte field in storage containing also a binary value. The M (Multiply fullword) instruction handles this problem. Let's consider the multiply instruction first (specially the size of the product): when two 32-bit operands are multiplied, the product requires 64-bits. Certainly the product cannot fit into the 1st-operand register! So it is not placed in a single register, but in a register-pair! The assembler requires that the register-pair has to be an even-odd pair!
The contents of the 1st-operand odd-register (the contents of the even-register are ignored) are multiplied by the data of the 2nd-operand field, a fullword (->> boundary!) in storage and the resulting product is placed in the 1st-operand register-pair. The operands are treated as 32-bit signed binary integers and the result is treated as a 64-bit signed binary integer. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules.

Condition codes
Condition code remains unchanged!

Graphical example



MH Multiply Halfword

Mnemonic	Operands
MH	R_1 , D_2 (X_2 , B_2)

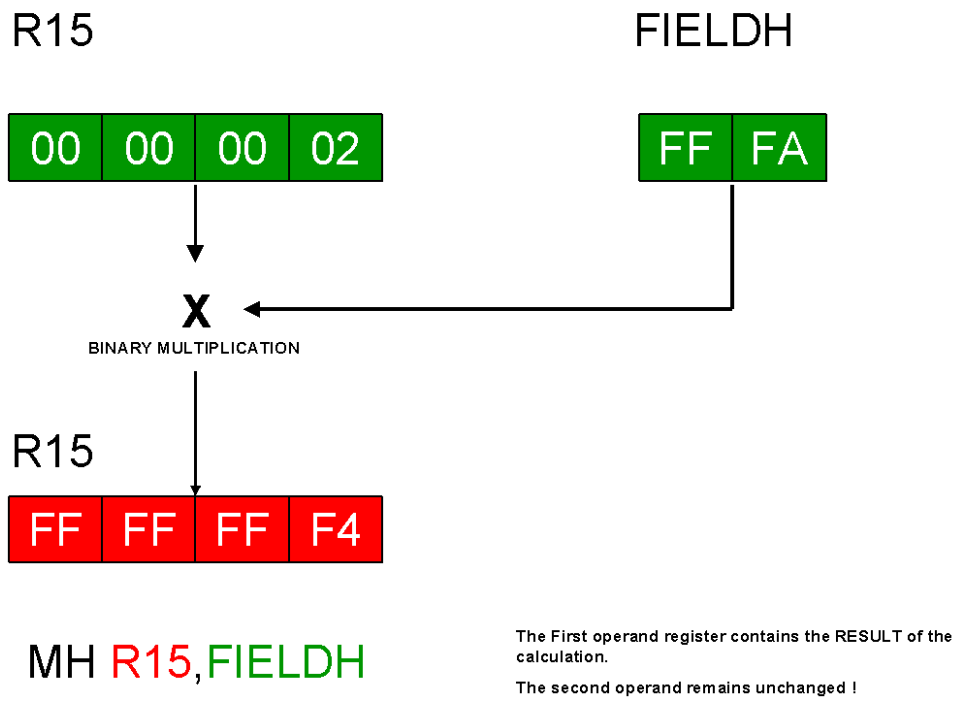
Machine format (RX)

4 C	R_1	X_2	B_2	D_2
0	8	12	16	20 31

The binary value of a register has to be multiplied by a two byte field in storage containing also a binary value. The MH (Multiply Halfword) instruction handles this problem. The contents of the 1st-operand register are multiplied by the data of the 2nd-operand field, a halfword (->> boundary!) in storage and the resulting product is placed in the 1st-operand register. Now we are multiplying a 32-bit operand by a 16-bit operand. The product contains 48 bits, but only the low-order 32 bits of the product are placed in the 1st-operand register. If the product requires more than 31 bits and the sign bit, the high-order bits are lost and the result is invalid. No overflow occurs!
The 2nd-operand field is treated as a 16-bit signed binary integer. The 1st-operand register and the result are treated as 32-bit signed binary integers. The 2nd-operand field remains unchanged. The calculation follows algebraic rules.

Condition codes
Condition code remains unchanged!

Graphical example

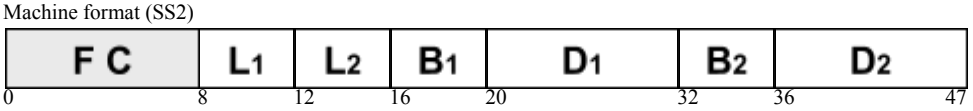


MP Multiply packed decimal data

Mnemonic | Operands

MP	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

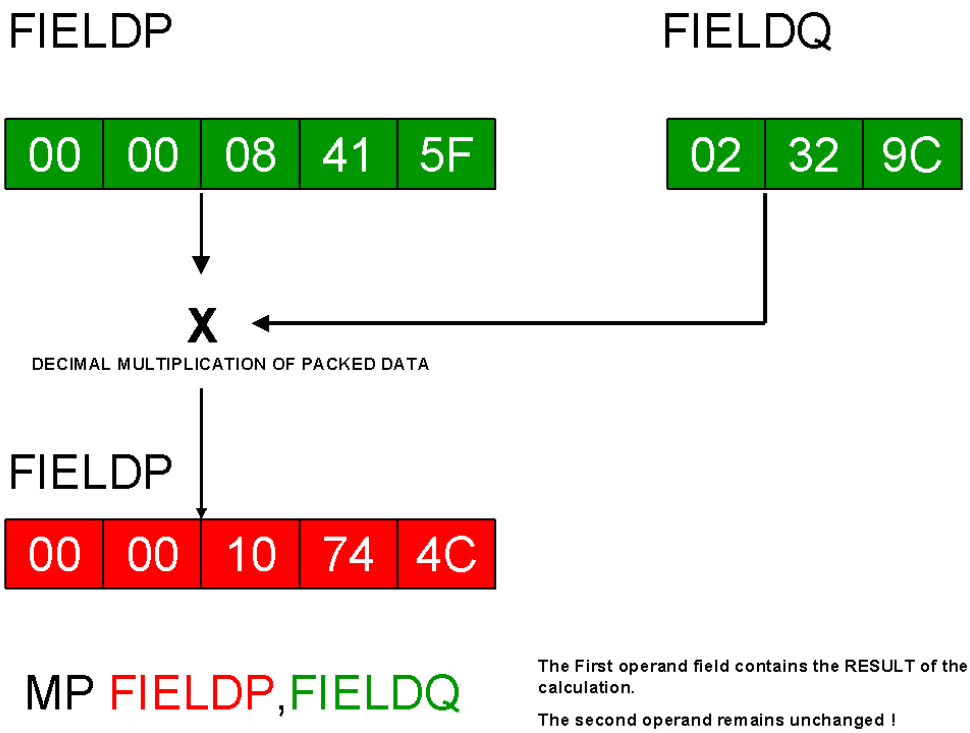
Machine format (SS2)



Two fields containing packed decimal data have to be multiplied with each other. The instruction MP (Multiply Packed decimal data) will do this in one step. The contents of the 1st-operand field (multiplicand) is multiplied with the contents of the 2nd-operand field (multiplier) and the result is placed in the 1st-operand field. The initial contents of both operand fields have to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The result is also in packed decimal format. The 2nd-operand field remains unchanged.
The multiplicand must have at least as many bytes of leading zeros as the number of bytes in the multiplier.
The calculation follows algebraic rules. The result is tested and a condition code is set. In case of an overflow the overflow digits are ignored.

Condition codes
Condition code remains unchanged!

Graphical example



MR Multiply register

Mnemonic	Operands
MR	R1 , R2

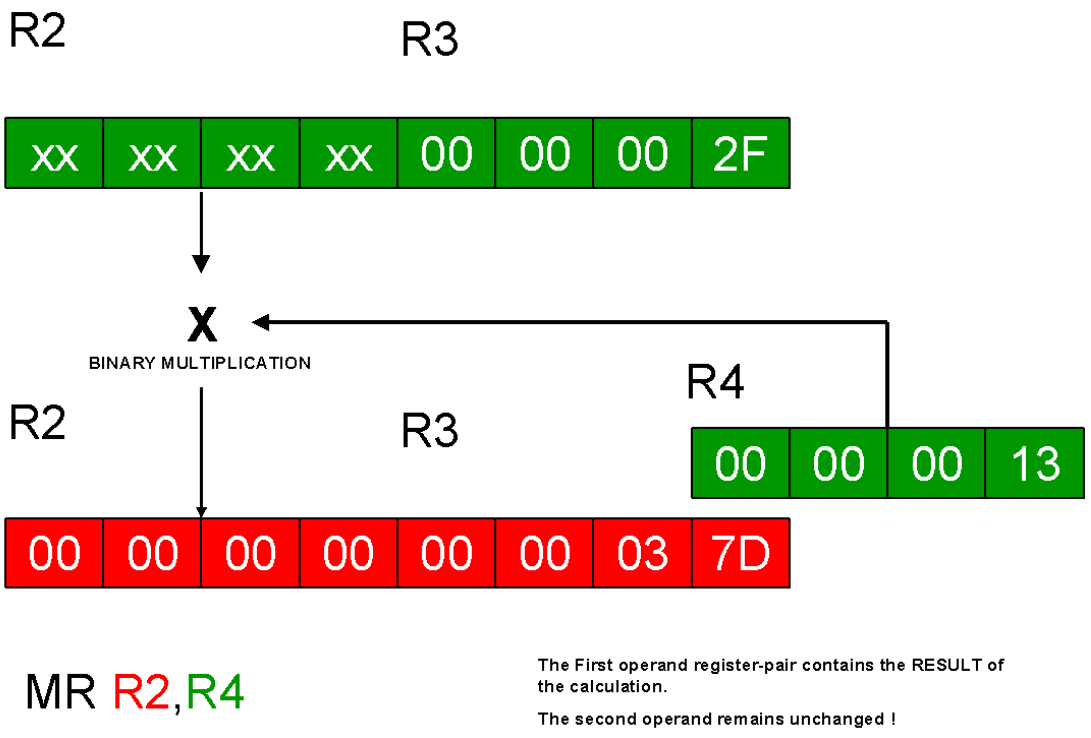
Machine format (RR)

1 C	R1	R2
0	8	12 15

To multiply the contents of one register by another register, the MR (Multiply Register) instruction is performed. The considerations about the size of the product and the 1st-operand register-pair are described in the M (Multiply fullword) instruction.
The contents of the 1st-operand odd-register (the contents of the even-register are ignored) are multiplied by the contents of the 2nd-operand register and the resulting product is placed in the 1st-operand register-pair. The operands are treated as 32-bit signed binary integers and the result is treated as a 64-bit signed binary integer. The 2nd-operand register remains unchanged.
The calculation follows algebraic rules.

Condition codes
Condition code remains unchanged!

Graphical example



MVC Move character

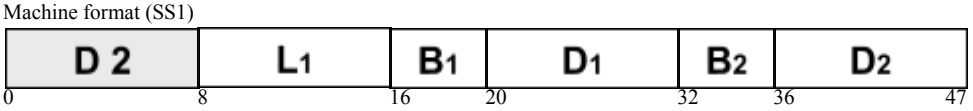
Mnemonic

MVC

Operands

$D_1 (L_1 , B_1) , D_2 (B_2)$

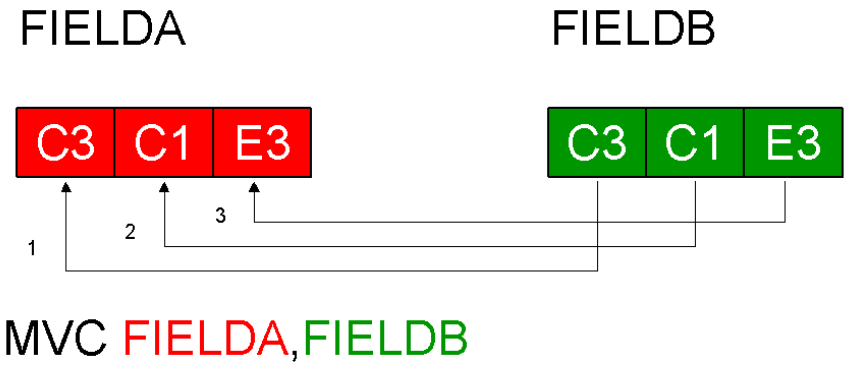
Machine format (SS1)



Data transfers between two storage locations are performed by the MVC (MoVe Character) instruction. This is one of the most frequently-used assembler instructions. It moves data from the source-field (2nd operand) into the target-field (1st operand). The maximum length which can be moved is 256 bytes of data. The length is defined by the 1st-operand field, a): explicitly as in MVC FIELD1(14),FIELD2, or b): implicitly using the length defined at field definition. If the specified length is less than the length of the source-field (2nd operand), the low-order bytes of data are lost. The length of the source-field is not of interest. Data movement proceeds from left to right, byte by byte. The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



The number of bytes moved is determined by the length of the 1st operand !

MVCL Move character long

Mnemonic | Operands

MVCL	R1 , R2

Machine format (RR)

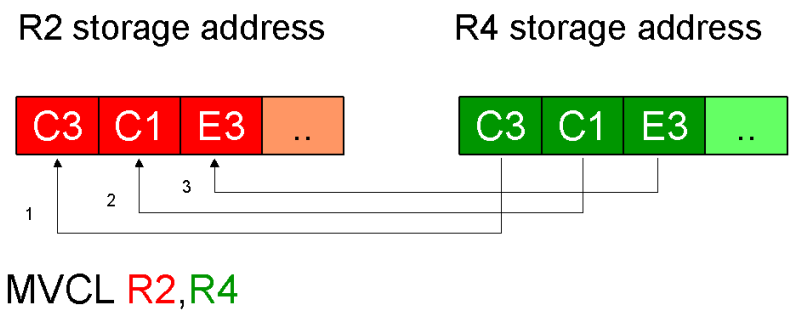
0 E	R1	R2

0 8 12 15

Data transfers between two large storage locations are performed by the MVCL (MoVe Character Long) instruction. It moves data from the source-field (designated by the 2nd-operand register-pair) into the target-field (designated by the 1st-operand register-pair). The maximum number which can be moved are 16777215 bytes of data. The instruction recommends even-odd pairs of registers.
Data movement proceeds from left to right, byte by byte, and stops as soon as the end of the target-field is reached. If source-field is shorter than the target-field, the remaining rightmost byte positions of the target-field are filled with padding bytes. If the source-field is larger than the target-field, the remaining bytes of the source-field are ignored.
The result is indicated in the condition code which may be checked subsequently. The source-field remains unchanged.
1)* operand lengths are equal
2)* 1st operand length is lower
3)* 1st operand length is higher
4)* destructive overlap, no movement performed

Condition codes
0 operand lengths are equal
1 1st operand length is lower than 2nd operand length
2 1st operand length is greater than 2nd operand length
3 destructive overlap, no movement performed

Graphical example

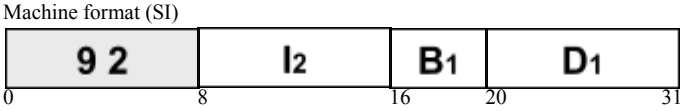


The number of bytes to be moved is in R3 (TO-LENGTH) and R5 (FROM-LENGTH) !

MVI Move immediate

Mnemonic	Operands
MVI	D_1 (B_1) , I_2

Machine format (SI)



To move a single byte of data, defined at program coding, into a storage location the MVI (MoVe Immediate) instruction is performed. The source-data (2nd operand) is moved into the target-field (1st operand). The single byte of the 2nd operand is also called the immediate operand. The immediate data may be specified as a): a character (EBCDIC code) operand as in MVI FIELD,C'A', b): a hexadecimal operand as in MVI FIELD,X'C1', or c): a binary operand as in MVI FIELD,B'11000001'.
The immediate data to be moved is contained within the instruction itself. The length of the target-field is not of interest. Only the 1st byte of the field will be overwritten.

Condition codes
Condition code remains unchanged!

Graphical example

MVN Move numeric

Mnemonic | Operands

MVN	$D_1 (L_1 , B_1) , D_2 (B_2)$

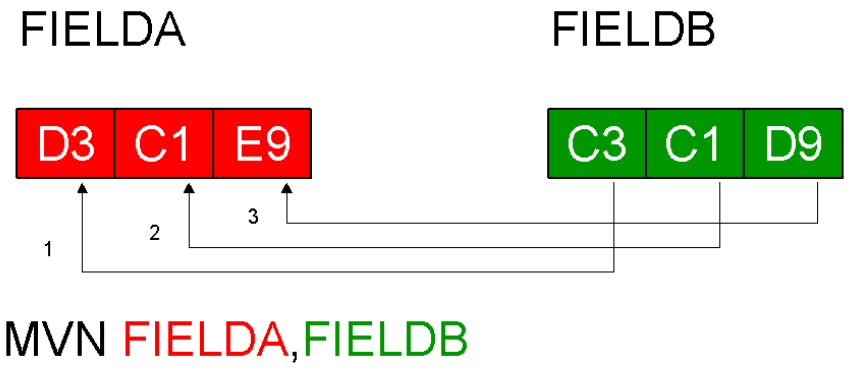
Machine format (SS1)

D 1	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To transfer the four low-order bits (numeric bits, numeric digit) between two storage locations the MVN (MoVe Numerics) instruction is performed. It moves the numeric data from the source-field (2nd operand) to the corresponding positions of the target-field (1st operand). The maximum length which can be handled is 256 bytes of data. The length is defined by the 1st operand, a): explicitly as in MVN FIELDX(24),FIELDZ, or b): implicitly using the length defined at field definition. If the specified length is less than the length of the source-field (2nd operand), the low-order bytes of data are lost. The length of the source-field is not of interest. Numeric digit movement proceeds from left to right, numeric digit by numeric digit. The source-field and the zone digits of the target-field remain unchanged.
See also the MVZ instruction.

Condition codes
Condition code remains unchanged!

Graphical example



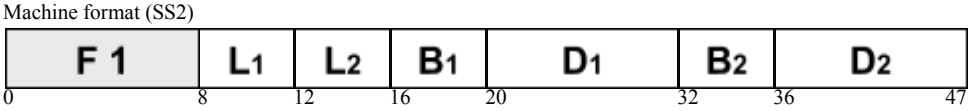
The number of numeric digits moved is determined by the length of the 1st operand !

MVO Move with offset

Mnemonic | Operands

MVO	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

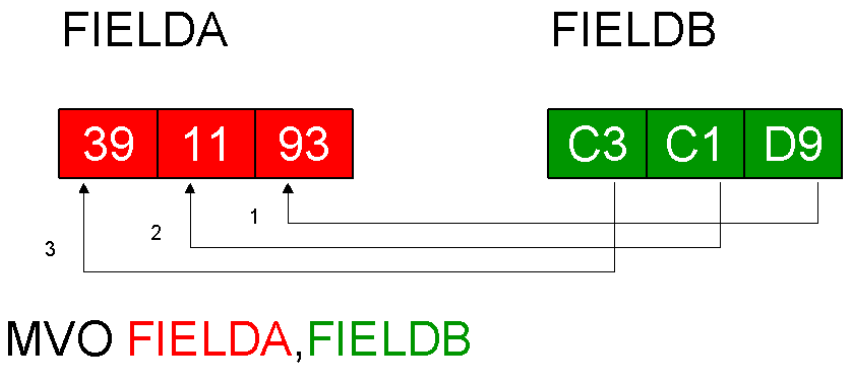
Machine format (SS2)



The last of the decimal move instructions, the MVO (MoVe with Offset) instruction, is the most complex. The data of the source-field (2nd operand) is moved and adjusted to the left of the low-order digit of the target-field (1st operand). The assembler must know the length of each operand field, which may be up to 16 (!) bytes long.
If the length of the 1st-operand (receiving) field is too small, the leftmost positions of the 2nd-operand (sending) field are truncated. If the length of the 1st-operand field is too large, the field is padded with zeros. Data movement proceeds from right to left, digit by digit. The source-field and the low-order digit of the target-field remain unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



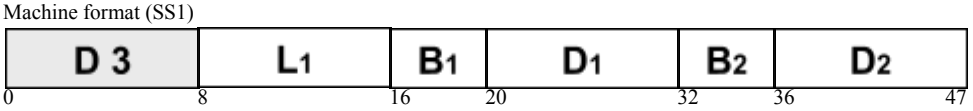
The last digit of the 1st operand remains unchanged !

MVZ Move zone

Mnemonic | Operands

MVZ	$D_1 (L_1 , B_1) , D_2 (B_2)$

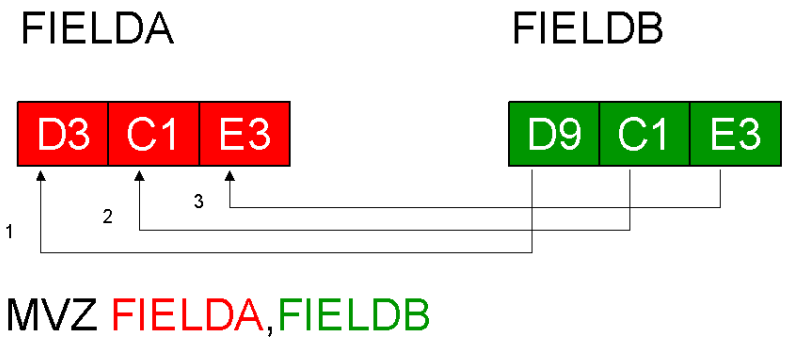
Machine format (SS1)



To transfer the four high-order bits (zone bits, zone digit) between two storage locations the MVZ (MoVe Zones) instruction is performed. It moves the zone data from the source-field (2nd operand) to the corresponding positions of the target-field (1st operand). The maximum length which can be handled is 256 bytes of data. The length is defined by the 1st operand, a): explicitly as in MVZ FIELDA(15),FIELDB, or b): implicitly using the length defined at field definition. If the specified length is less than the length of the source-field (2nd operand), the low-order bytes of data are lost. The length of the source-field is not of interest. Zone digit movement proceeds from left to right, zone digit by zone digit. The source-field and the numeric digits of the target-field remain unchanged.
See also the MVN instruction.

Condition codes
Condition code remains unchanged!

Graphical example



The number of zone digits moved is determined by the length of the 1st operand !

N And fullword

Mnemonic | Operands

N	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

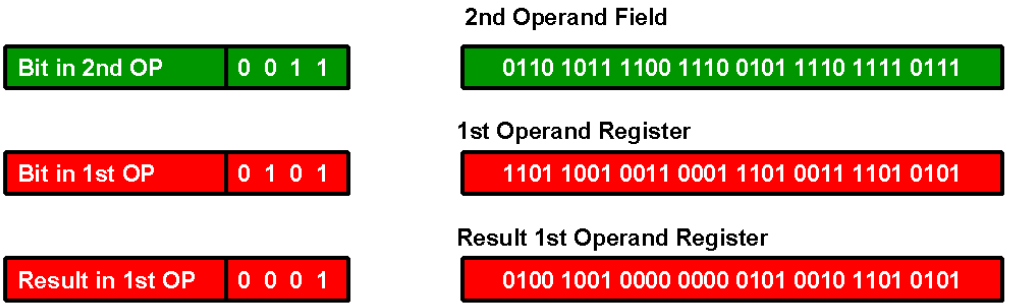
5 4	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To set single bits of a register to 'zero', the N (aNd fullword) instruction is performed. The 'one'-bits of the 1st-operand register are changed to 'zero', if the corresponding bits of the 2nd-operand field, a fullword (->> boundary!) in storage, are 'zero'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the NR instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	

Graphical example



NC And character

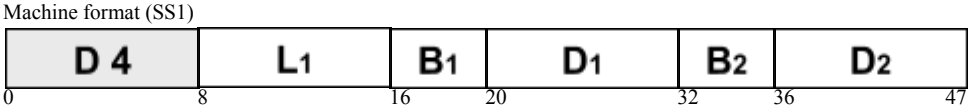
Mnemonic

NC

Operands

$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

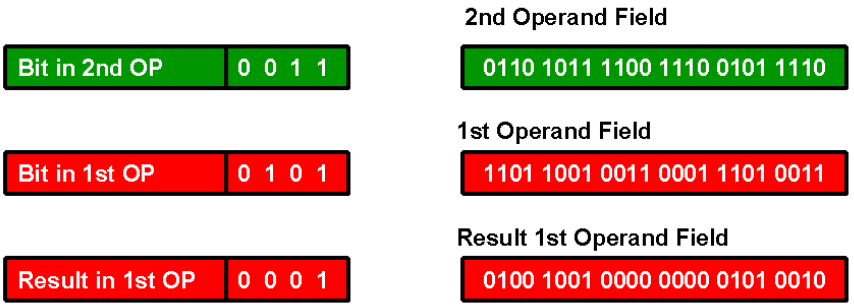


To set single bits of a storage field to 'zero', the NC (aNd Character) instruction is performed. The 'one'-bits of the 1st-operand field are changed to 'zero', if the corresponding bits of the 2nd-operand field are 'zero'. In all other cases the bits of the 1st-operand field remain unchanged.
The result is indicated in the condition code which may be checked subsequently.is indicated in the condition code which may be checked subsequently.

Condition codes

0	**= 0**	1st operand field is zero
1	**!= 0**	1st operand field is not zero
2	**not used!**	
3	**not used!**	

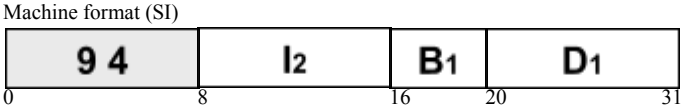
Graphical example



NI And immediate

Mnemonic	Operands
NI	D_1 (B_1) , I_2

Machine format (SI)

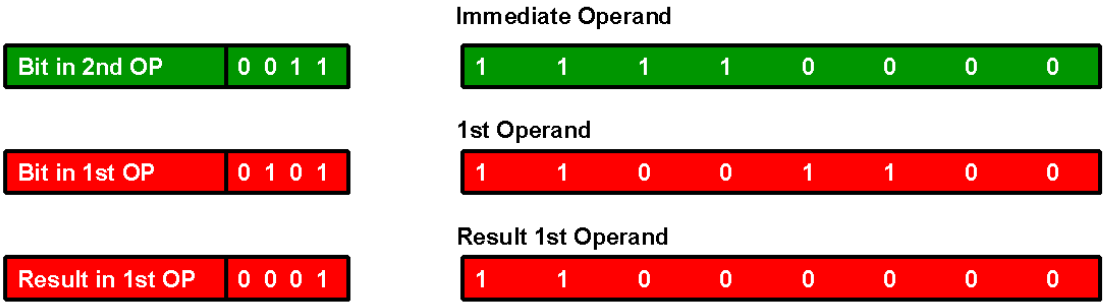


To set the bits of a single byte of data to 'zero', the NI (aNd Immediate) instruction is performed. The 'one'-bits of the 1st-operand field are changed to 'zero', if the corresponding bits of the 2nd-operand field are 'zero'. In all other cases the bits of the 1st-operand field remain unchanged.
The 2nd operand is also called the immediate operand. The immediate data, defined at program coding, is contained within the instruction itself. The length of the 1st-operand field is not of interest. Only the 1st byte of the field is considered. The result is indicated in the condition code which may be checked subsequently.est. Only the 1st byte of the field is considered.

Condition codes

0	**= 0**	1st operand byte is zero
1	**!= 0**	1st operand byte is not zero
2	**not used!**	
3	**not used!**	

Graphical example



NR And register

Mnemonic	Operands
NR	R_1 , R_2

Machine format (RR)

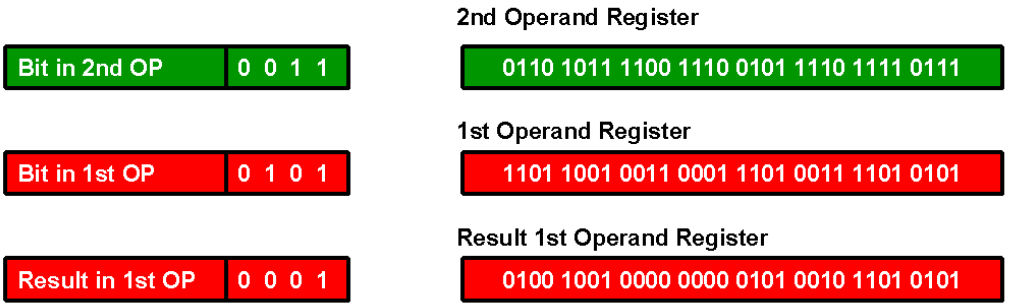
1 4	R_1	R_2
0	8	12 15

To set single bits of a register to 'zero', the NR (aNd Register) instruction is performed. The 'one'-bits of the 1st-operand register are changed to 'zero', if the corresponding bits of the 2nd-operand register are 'zero'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the N instruction.ain unchanged.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	

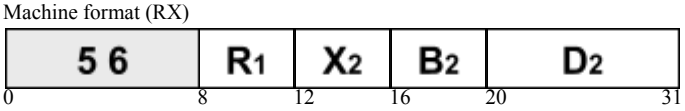
Graphical example



O Or fullword

Mnemonic	Operands
O	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

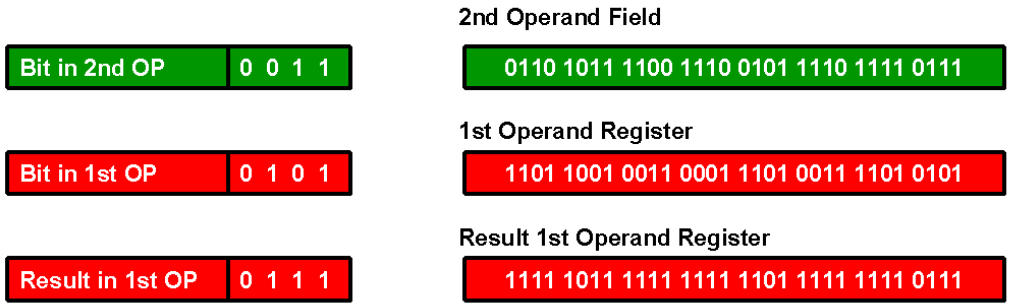


To set single bits of a register to 'one', the O (Or fullword) instruction is performed. The 'zero'-bits of the 1st-operand register are changed to 'one', if the corresponding bits of the 2nd-operand field, a fullword (->> boundary!) in storage, are 'one'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the OR instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	

Graphical example



OC Or character

Mnemonic | Operands

OC	$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

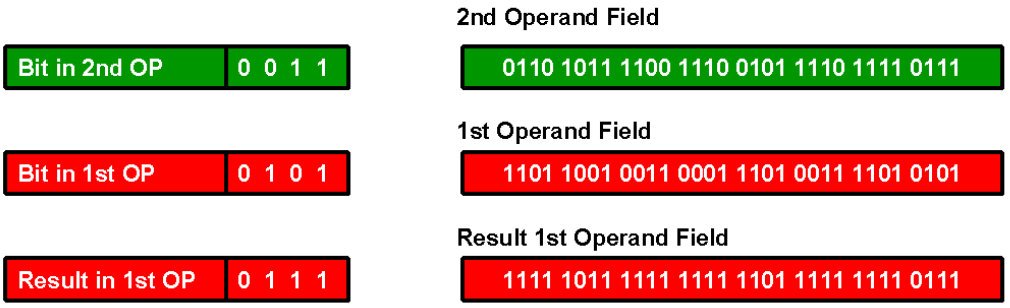
D 6	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To set single bits of a storage field to 'one', the OC (Or Character) instruction is performed. The 'zero'-bits of the 1st-operand field are changed to 'one', if the corresponding bits of the 2nd-operand field are 'one'. In all other cases the bits of the 1st-operand field remain unchanged.
The result is indicated in the condition code which may be checked subsequently.

Condition codes

0	**= 0**	1st operand field is zero
1	**!= 0**	1st operand field is not zero
2	**not used!**	
3	**not used!**	

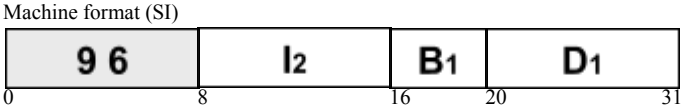
Graphical example



OI Or immediate

Mnemonic	Operands
OI	D_1 (B_1) , I_2

Machine format (SI)

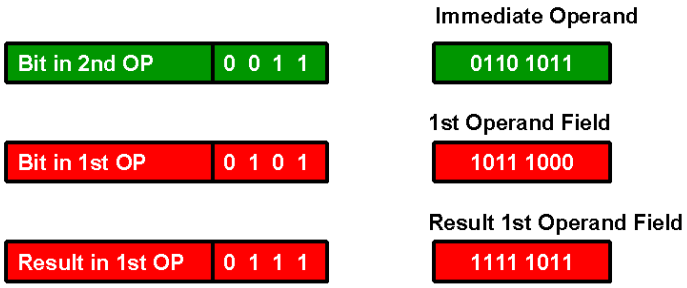


To set the bits of a single byte of data to 'one', the OI (Or Immediate) instruction is performed. The 'zero'-bits of the 1st-operand field are changed to 'one', if the corresponding bits of the 2nd-operand field are 'one'. In all other cases the bits of the 1st-operand field remain unchanged.
The 2nd operand is also called the immediate operand. The immediate data, defined at program coding, is contained within the instruction itself. The length of the 1st-operand field is not of interest. Only the 1st byte of the field is considered. The result is indicated in the condition code which may be checked subsequently.

Condition codes

0	**= 0**	1st operand byte is zero
1	**!= 0**	1st operand byte is not zero
2	**not used!**	
3	**not used!**	

Graphical example



OR Or register

Mnemonic	Operands
OR	R_1 , R_2

Machine format (RR)

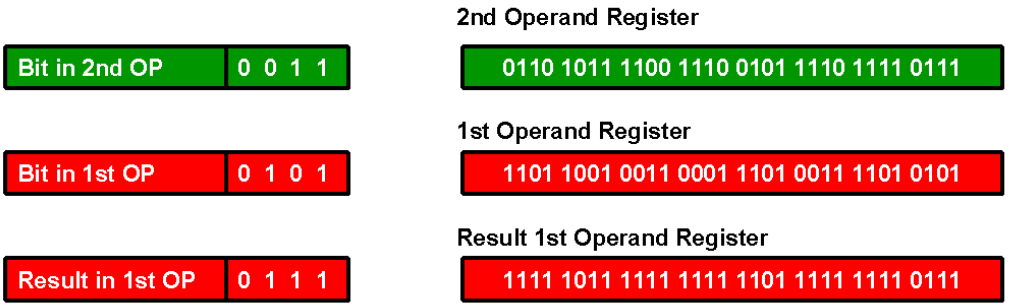
1 6	R_1	R_2
0	8	12 15

To set single bits of a register to 'one', the OR (Or Register) instruction is performed. The 'zero'-bits of the 1st-operand register are changed to 'one', if the corresponding bits of the 2nd-operand register are 'one'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the O instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	

Graphical example



PACK Pack

Mnemonic | Operands

PACK	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

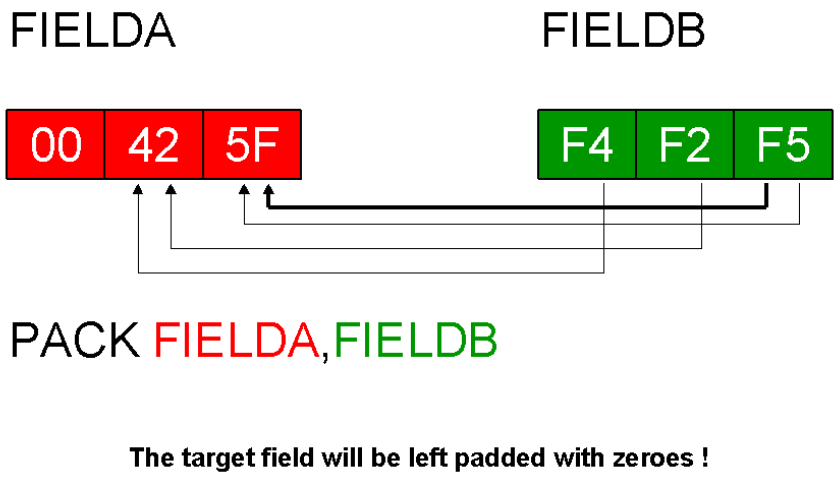
Machine format (SS2)

F 2	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

The PACK instruction converts decimal data to packed format. The instruction takes the data from the source-field (2nd operand) and packs it to the target-field (1st operand). The assembler must know the length of each operand field, which may be up to 16 (!) bytes long. The instruction proceeds from right to left.
If the length of the 1st-operand (receiving) field is too small, the leftmost positions of the 2nd-operand (sending) field are truncated. If the length of the 1st-operand field is too large, the field is padded with zeros.
After execution, the low-order digit of the target-field signals the sign of the packed decimal value (Positive: A, C, E, F ; Negative: B, D). The source-field remains unchanged

Condition codes
Condition code remains unchanged!

Graphical example



The target field will be left padded with zeroes !

S Subtract fullword

Mnemonic	Operands
S	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

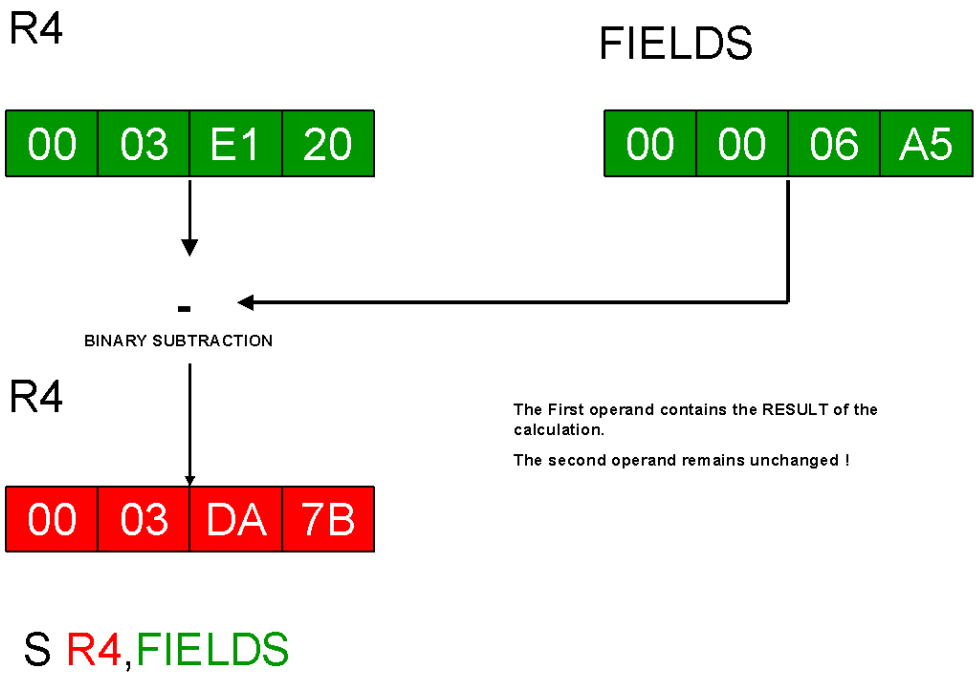
5 B	R_1	X_2	B_2	D_2
0	8	12	16	20 31

A four byte field in storage containing a binary value has to be subtracted from the binary value of a register. The S (Subtract fullword) instruction handles this problem. The data of the 2nd-operand field, a fullword (->> boundary!) in storage, is subtracted from the contents of the 1st-operand register and the result is placed in the 1st-operand register. The operands and the result are treated as 32-bit signed binary integers. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

Graphical example



SH Subtract Halfword

Mnemonic | Operands

SH	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

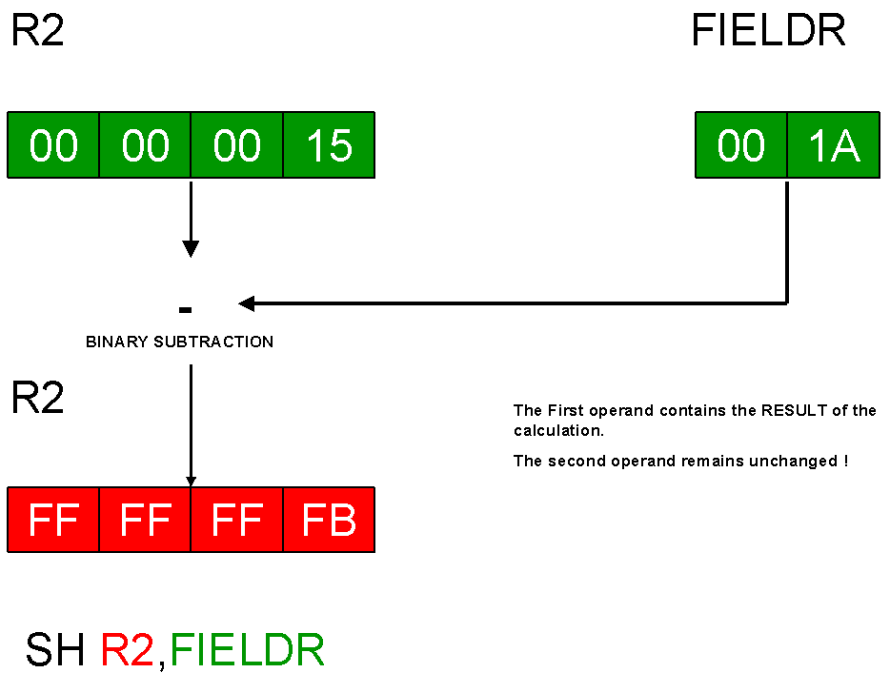
4 B	R_1	X_2	B_2	D_2
0	8	12	16	20 31

A two byte field in storage containing a binary value has to be subtracted from the binary value of a register. The problem can be solved by using the SH (Subtract Halfword) instruction. The data of the 2nd-operand field, a halfword (->>> boundary!) in storage, is subtracted from the contents of the 1st-operand register and the result is placed in the 1st-operand register. The 2nd-operand field is treated as a 16-bit signed binary integer. The 1st-operand register and the result are treated as 32-bit signed binary integers. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

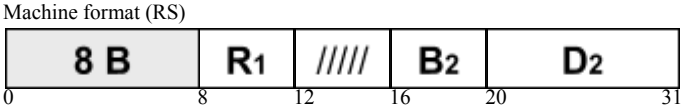
Graphical example



SLA Shift left single arithmetic

Mnemonic	Operands
SLA	R_1 , D_2 (B_2)

Machine format (RS)

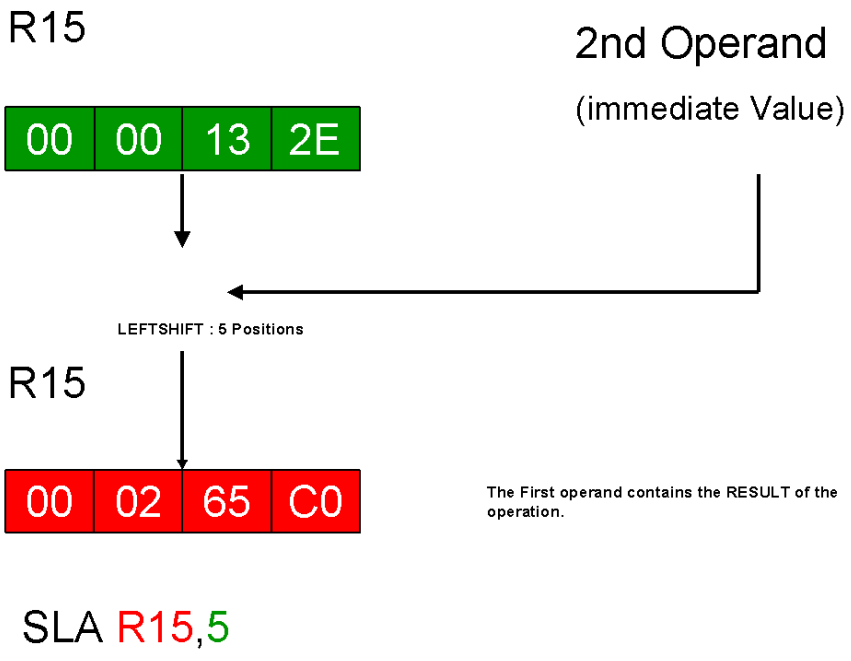


The data of a register has to be shifted to the left by a number of positions. The SLA (Shift Left single Arithmetic) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. The 1st-operand register is treated as a 32-bit signed binary integer.
If the binary value of the 1st-operand register is shifted to the left by one position, the value is doubled. During the left-shift zeros are inserted on the right of the 1st-operand register.
The sign bit is not shifted! The result is tested and a condition code is set. If one or more bits unlike the sign bit are shifted out of the leftmost position, an overflow occurs and additionally the condition code is set to 3.
See also the SLL instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**< 0**	1st operand register is negative
2	**> 0**	1st operand register is positive
3	**overflow**	a bit unequal the sign bit is shifted out to the left

Graphical example



SLDA Shift left double arithmetic

Mnemonic	Operands
SLDA	R1 , D2 (B2)

Machine format (RS)

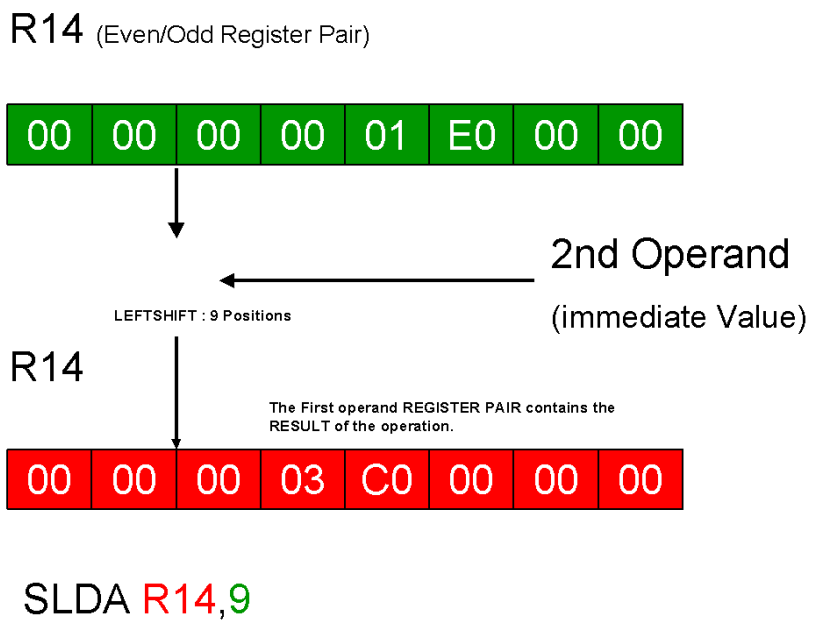
8 F	R1	/////	B2	D2
0	8	12	16	20 31

The data of a register-pair has to be shifted to the left by a number of positions. The SLDA (Shift Left Double Arithmetic) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. The 1st-operand register-pair is treated as a 64-bit signed binary integer. The instruction recommends an even-odd pair of registers.
If the binary value of the register-pair is shifted to the left by one position, the value is doubled. During the left-shift zeros are inserted on the right of the register-pair.
The sign bit is not shifted! The result is tested and a condition code is set. If one or more bits unlike the sign bit are shifted out of the leftmost position, an overflow occurs and additionally the condition code is set to 3.
See also the SLDL instruction.

Condition codes

0	**= 0**	1st operand register pair is zero
1	**< 0**	1st operand register pair is negative
2	**> 0**	1st operand register pair is positive
3	**overflow**	a bit unequal the sign bit is shifted out to the left

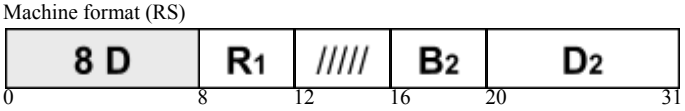
Graphical example



SLDL Shift left double logical

Mnemonic	Operands
SLDL	R_1 , D_2 (B_2)

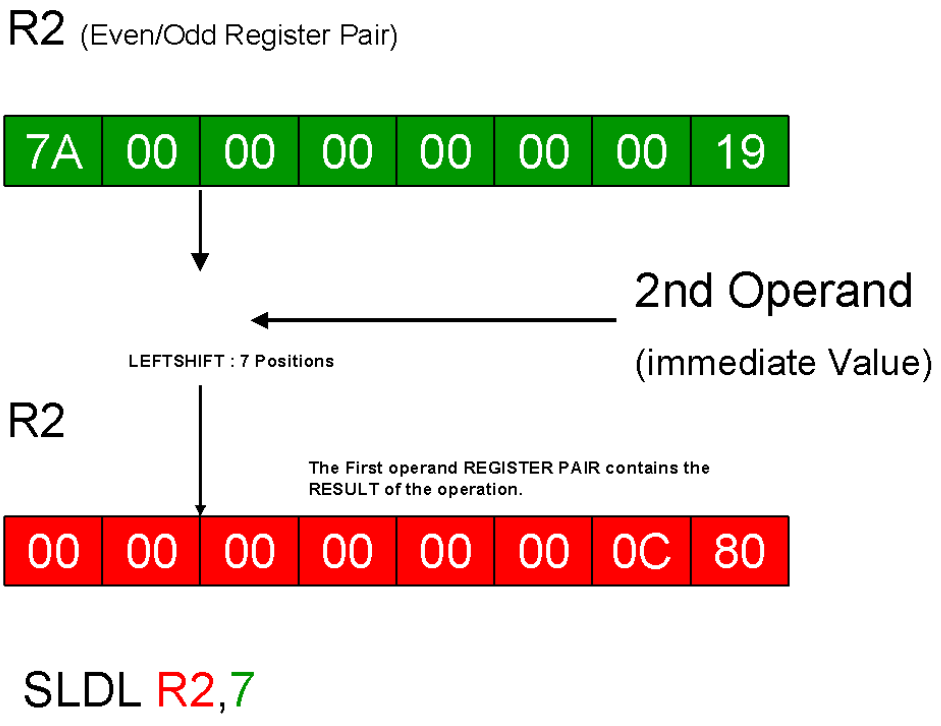
Machine format (RS)



The data of a register-pair has to be shifted to the left by a number of positions. The SLDL (Shift Left Double Logical) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. All 64 bits of the 1st-operand register-pair are included in the left-shift. The instruction recommends an even-odd pair of registers.
If the binary value of the register-pair is shifted to the left by one position, the value is doubled. During the left-shift zeros are inserted on the right of the register-pair.
Bits shifted out of the leftmost position are lost.
See also the SLDA instruction.

Condition codes
Condition code remains unchanged!

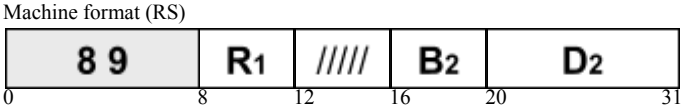
Graphical example



SLL Shift left single logical

Mnemonic	Operands
SLL	R_1 , D_2 (B_2)

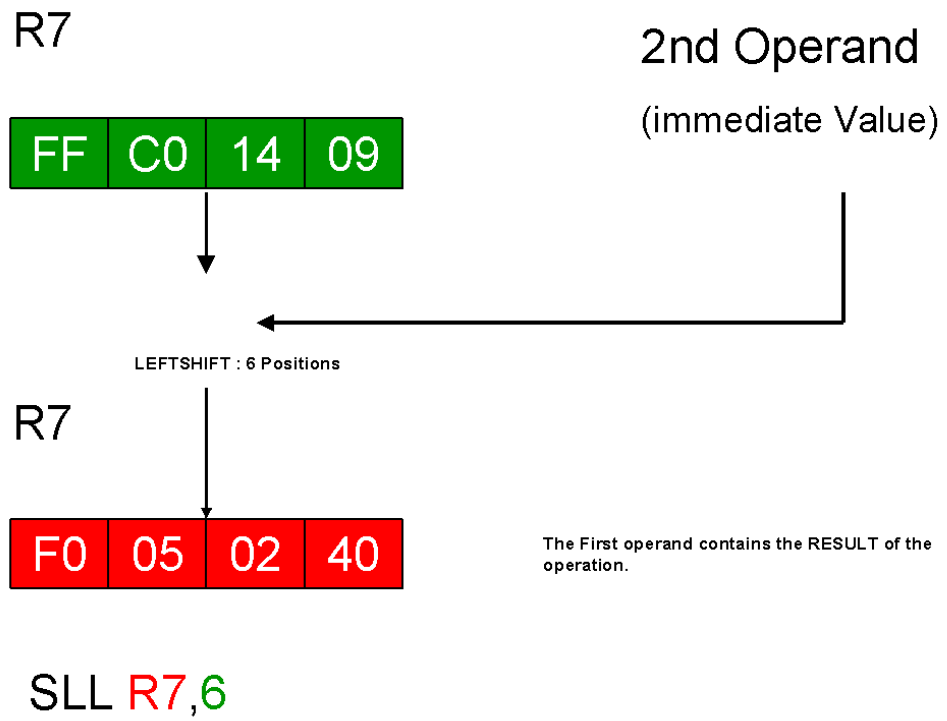
Machine format (RS)



The data of a register has to be shifted to the left by a number of positions. The SLL (Shift Left single Logical) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. All 32 bits of the 1st-operand register are included in the left-shift.
If the binary value of the 1st-operand register is shifted to the left by one position, the value is doubled. During the left-shift zeros are inserted on the right of the 1st-operand register.
Bits shifted out of the leftmost position are lost.
See also the SLA instruction.

Condition codes
Condition code remains unchanged!

Graphical example



SP Subtract packed decimal data

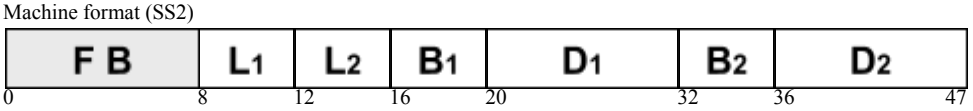
Mnemonic

SP

Operands

$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

Machine format (SS2)

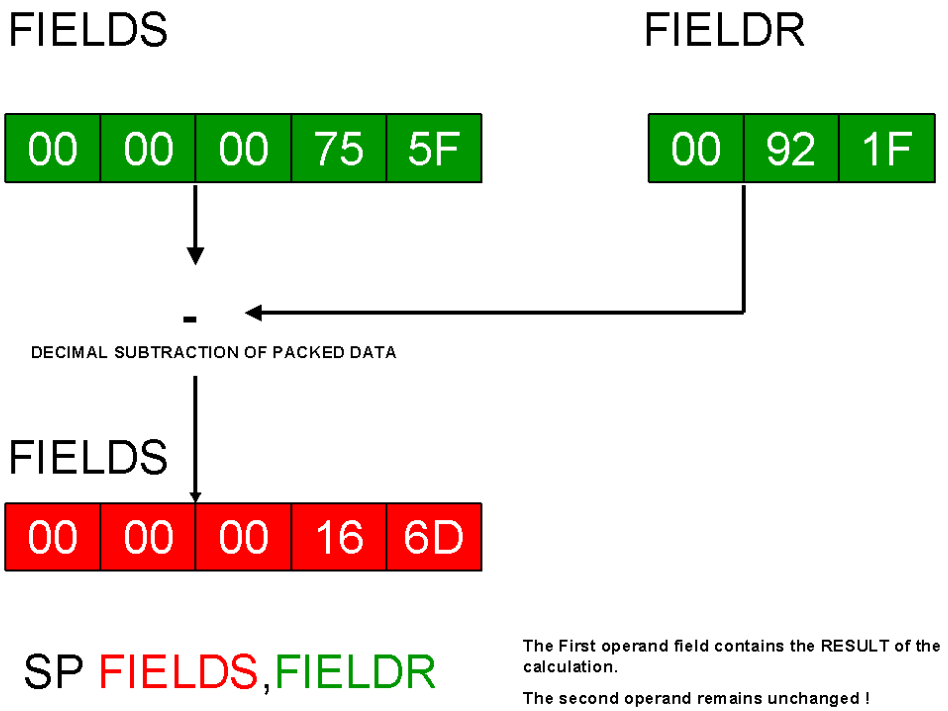


Two fields containing packed decimal data have to be subtracted from each other. The instruction SP (Subtract Packed decimal data) will do this in one step. The contents of the 2nd-operand field are subtracted from the contents of the 1st-operand field and the result is placed in the 1st-operand field. The initial contents of both operand fields have to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The result is also in packed decimal format. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. In case of an overflow the overflow digits are ignored and additionally the condition code is set to 3.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand field

Graphical example



SR Subtract register

Mnemonic | Operands

SR	R_1 , R_2

Machine format (RR)

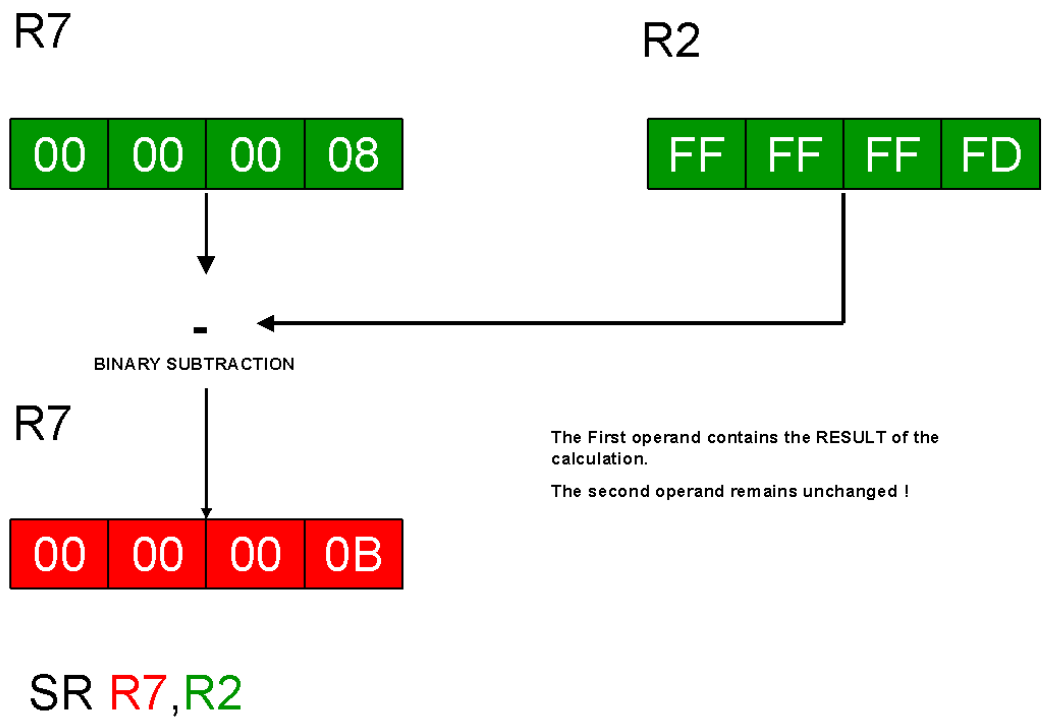
1 B	R_1	R_2
0	8	12 15

To subtract the contents of one register from another register, the SR (Subtract Register) instruction is performed. The contents of the 2nd-operand register are subtracted from the contents of the 1st-operand register and the result is placed in the 1st-operand register. The operands and the result are treated as 32-bit signed binary integers. The 2nd-operand register remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. If the result does not fit into the 1st-operand register an overflow occurs and additionally the condition code is set to 3.

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand register

Graphical example



SRA Shift right single arithmetic

Mnemonic | Operands

SRA	R_1 , D_2 (B_2)

Machine format (RS)

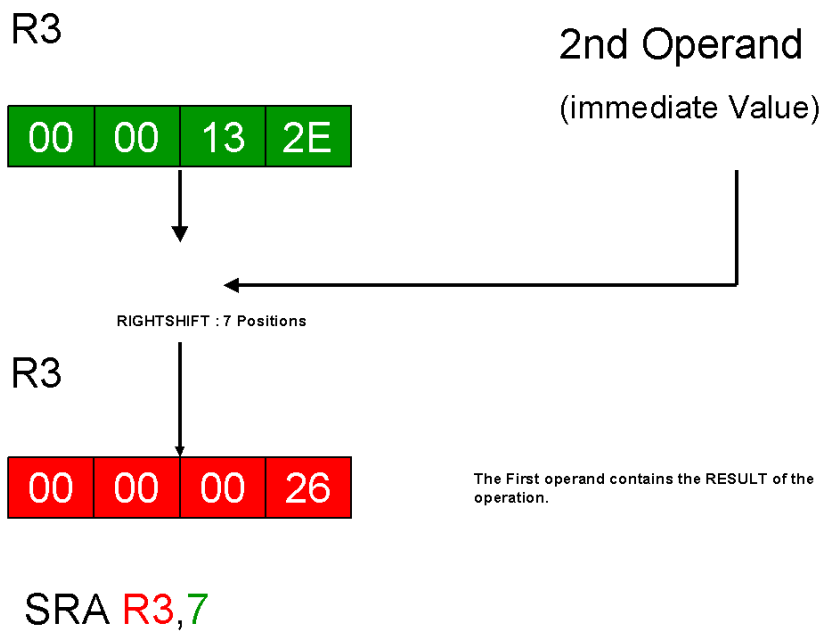
8 A	R_1	/////	B_2	D_2
0	8	12	16	20 31

The data of a register has to be shifted to the right by a number of positions. The SRA (Shift Right single Arithmetic) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. The 1st-operand register is treated as a 32-bit signed binary integer.
If the binary value of the 1st-operand register is shifted to the right by one position, the value is cut in halve. During the right-shift, bits equal to the sign bit are inserted on the left of the 1st-operand register.
Only the 31 rightmost bits are shifted. The sign bit (first bit) remains unchanged! The result is tested and a condition code is set. Bits shifted out of the rightmost position are lost.
See also the SRL instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**< 0**	1st operand register is negative
2	**> 0**	1st operand register is positive
3	**not used!**	

Graphical example



SRDA Shift right double arithmetic

Mnemonic	Operands
SRDA	R1 , D2 (B2)

Machine format (RS)

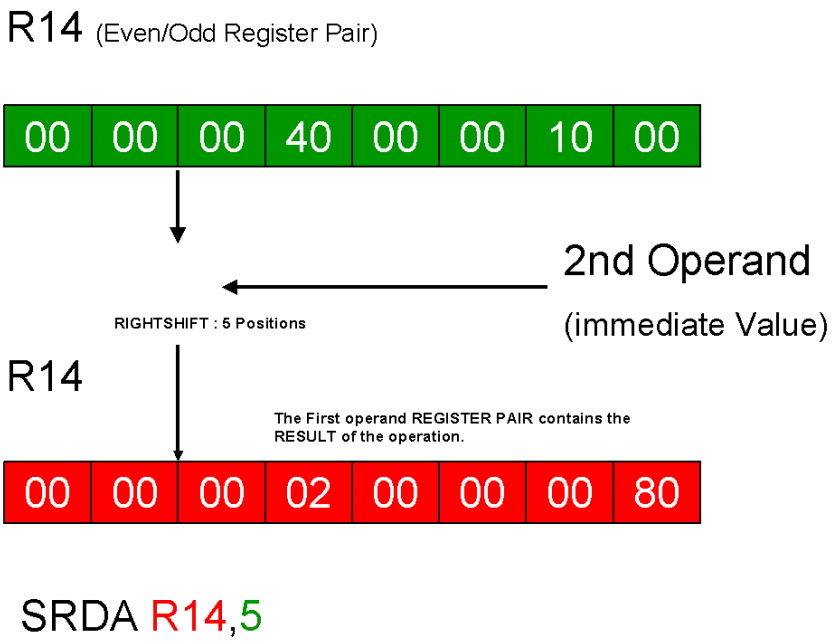
8 E	R1	/////	B2	D2
0	8	12	16	20 31

The data of a register-pair has to be shifted to the right by a number of positions. The SRDA (Shift Right Double Arithmetic) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. The 1st-operand register-pair is treated as a 64-bit signed binary integer. The instruction recommends an even-odd pair of registers.
If the binary value of the register-pair is shifted to the right by one position, the value is cut in halve. During the right-shift, bits equal to the sign bit are inserted on the left of the register-pair.
Only the 63 rightmost bits are shifted. The sign bit (first bit of the even register) remains unchanged! The result is tested and a condition code is set. Bits shifted out of the rightmost position are lost.
See also the SRDL instruction.

Condition codes

0	**= 0**	1st operand register pair is zero
1	**< 0**	1st operand register pair is negative
2	**> 0**	1st operand register pair is positive
3	**not used!**	

Graphical example



SRDA R14,5

SRDL Shift left double logical

Mnemonic	Operands
SRDL	R1 , D2 (B2)

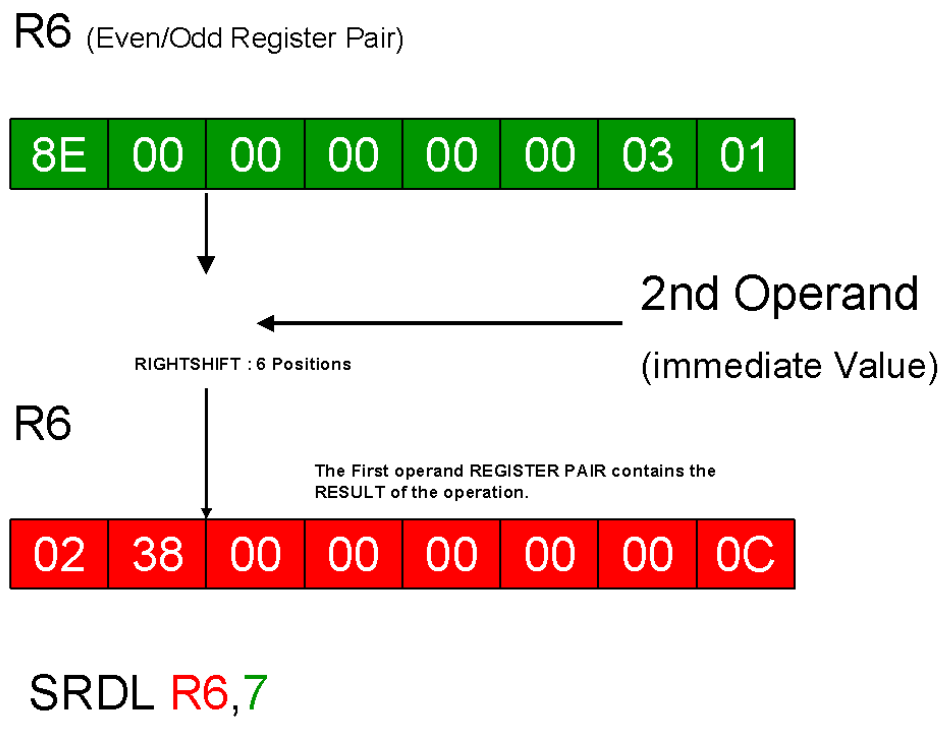
Machine format (RS)

8 C	R1	/////	B2	D2
0	8	12	16	20 31

The data of a register-pair has to be shifted to the right by a number of positions. The SRDL (Shift Right Double Logical) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. All 64 bits of the 1st-operand register-pair are included in the right-shift. The instruction recommends an even-odd pair of registers.
If the binary value of the register-pair is shifted to the right by one position, the value is cut in halve. During the right-shift zeros are inserted on the left of the register-pair.
Bits shifted out of the rightmost position are lost.
See also the SRDA instruction.

Condition codes
Condition code remains unchanged!

Graphical example



SRL Shift right single logical

Mnemonic	Operands
SRL	R_1 , D_2 (B_2)

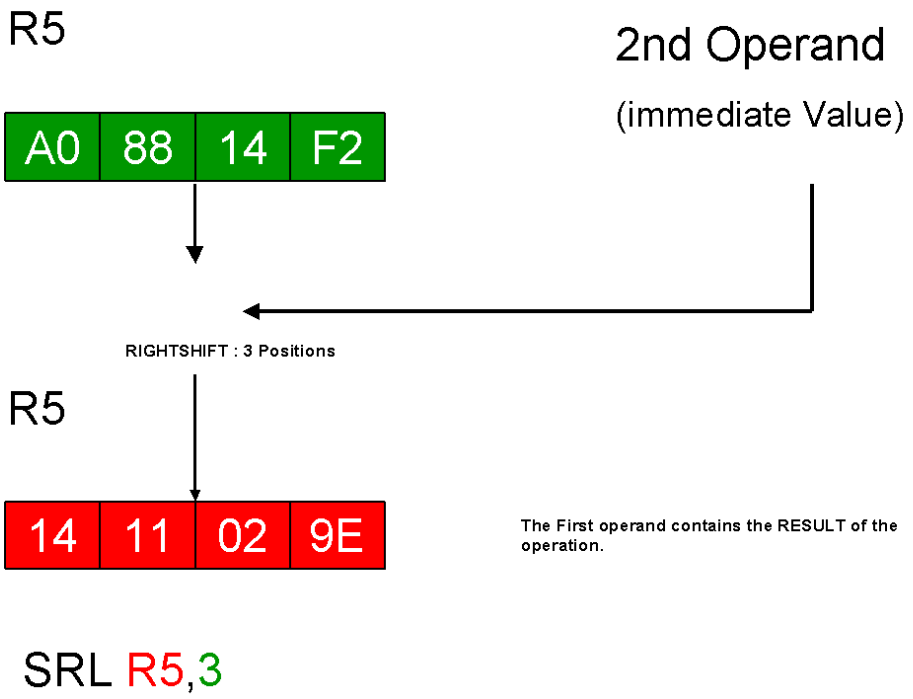
Machine format (RS)

8 8	R_1	/////	B_2	D_2
0	8	12	16	20 31

The data of a register has to be shifted to the right by a number of positions. The SRL (Shift Right single Logical) instruction handles this problem. The number of positions (maximum 63) to be shifted is given in the immediate 2nd-operand, also called as shift value. All 32 bits of the 1st-operand register are included in the right-shift.
If the binary value of the 1st-operand register is shifted to the right by one position, the value is cut in halve. During the right-shift zeros are inserted on the left of the 1st-operand register.
Bits shifted out of the rightmost position are lost.
See also the SRA instruction.ft of the 1st-operand register.
The sign bit is not shifted!

Condition codes
Condition code remains unchanged!

Graphical example



SRP Shift and round packed decimal data

Mnemonic | Operands

SRP	$D_1 (L_1 , B_1) , D_2 (B_2) , I_3$

Machine format (SS)

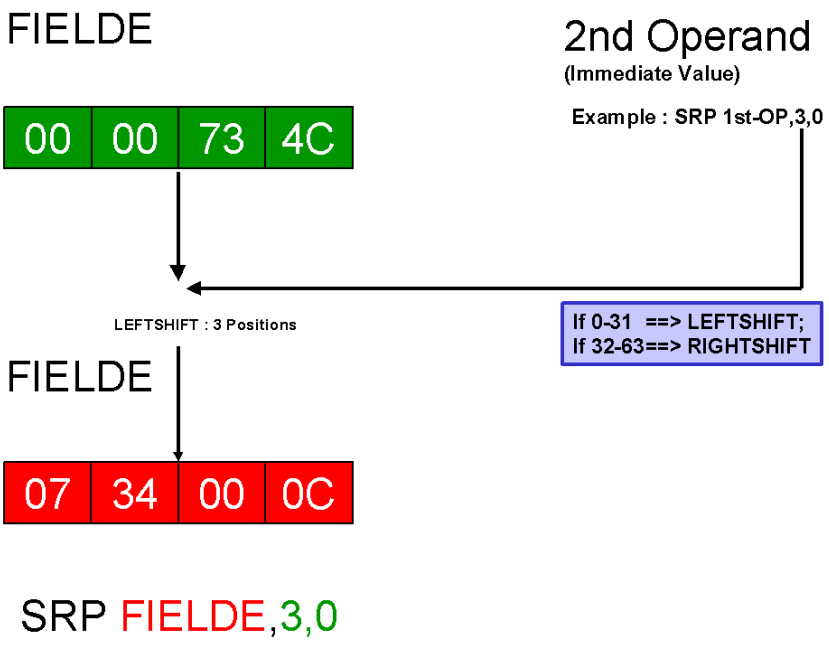
F 0	L_1	I_3	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

To cut and round a packed decimal value, the SRP (Shift and Round Packed decimal data) instruction is performed. The decimal digits of the 1st-operand field are shifted in the direction specified by the 2nd-operand address (only the 6 rightmost bits!). Additionally, if shifting to the right is specified, the result value of the 1st-operand field is rounded by the rounding factor (I3). It is not used in the leftshift.
The instruction can be used for left-shifting up to 31 positions (B'011111') and right-shifting up to 32 positions (B'100000').
The initial contents of the 1st-operand field has to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The result is also in packed decimal format.
The result is tested and a condition code is set. In case of an overflow the overflow digits are ignored and additionally the condition code is set to 3.

Condition codes

0	**= 0**	1st operand field is zero
1	**< 0**	1st operand field is negative
2	**> 0**	1st operand field is positive
3	**overflow**	result doesn't fit into the 1st operand field

Graphical example



ST Store fullword

Mnemonic	Operands
ST	R_1 , D_2 (X_2 , B_2)

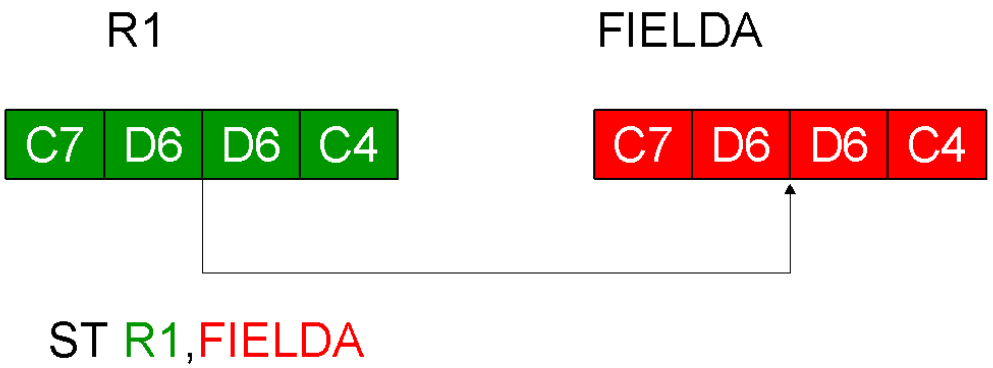
Machine format (RX)

5 0	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move the four bytes of binary data from a register to a storage location, the ST (STore fullword) instruction is performed. The contents of the source-register (1st operand) are stored in the target-field (2nd operand). The target-field has to be a fullword (->> boundary!) in storage. The source-register remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



STC Store character

Mnemonic | Operands

STC	R_1 , D_2 (X_2 , B_2)

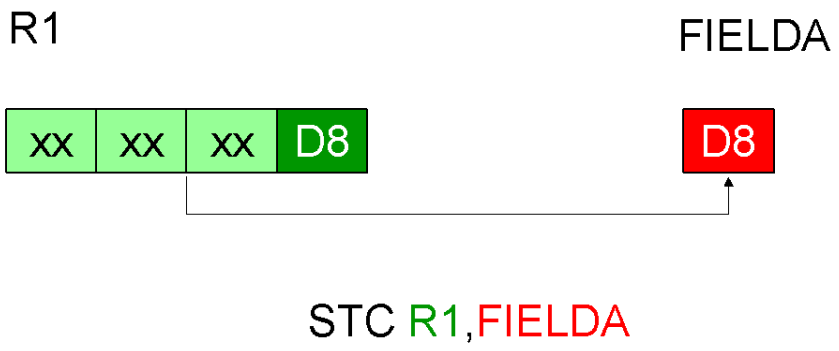
Machine format (RX)

4 2	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move the low-order byte of binary data from a register to a storage location, the STC (STore Character) instruction is performed. The contents of the low-order byte of the source-register (1st operand) are stored in the target-field (2nd operand). The three high-order bytes of the source-register are ignored. The source-register remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



xx - these bytes are ignored !

STCM Store character under mask

Mnemonic | Operands

STCM	R1 , M3 , D2 (B2)

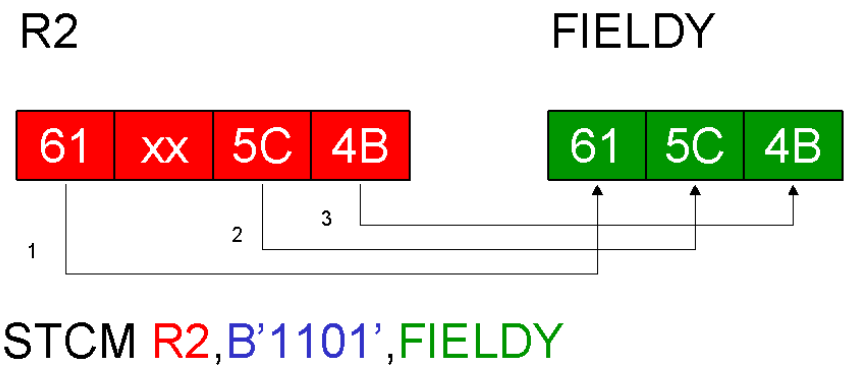
Machine format (RS)

B E	R1	M3	B2	D2
0	8	12	16	20 31

The STCM (STore Characters under Mask) instruction is an additional instruction to store one or more bytes of data from wanted positions in a register in a storage field. The data of the source-register (1st operand), bytes indicated by the 4-bit mask (3rd operand), are stored at the target-location (2nd operand). The length of the target-field is given by the number of indicated bytes. The source-register remains unchanged.
No boundary is wanted! See also the ST, STH and STC instructions.

Condition codes
Condition code remains unchanged!

Graphical example



Controlled by the 3rd operand MASK : Here = '1101'

STH Store Halfword

Mnemonic	Operands
STH	R_1 , D_2 (X_2 , B_2)

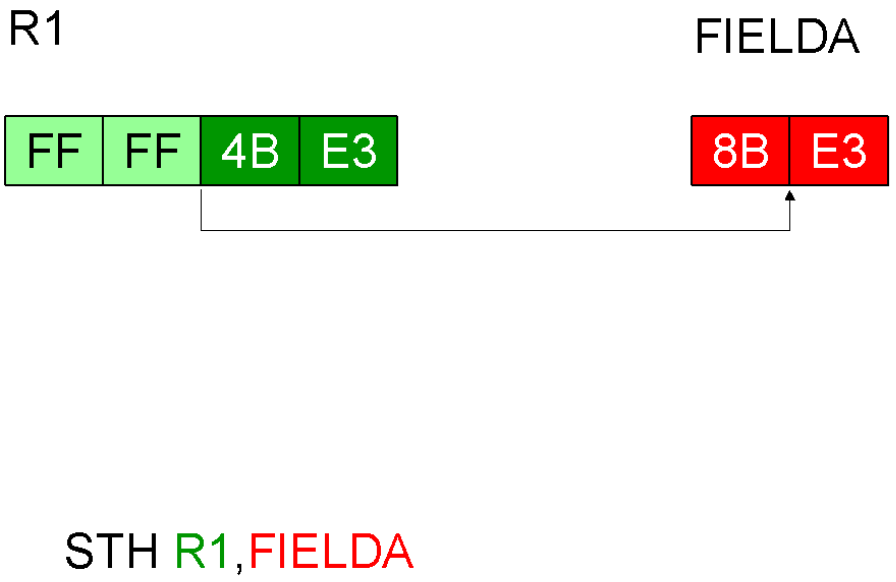
Machine format (RX)

4 0	R_1	X_2	B_2	D_2
0	8	12	16	20 31

To move the two low-order bytes of binary data from a register to a storage location, the STH (STore Halfword) instruction is performed. The contents of the two low-order bytes of the source-register (1st operand) are stored in the target-field (2nd operand). The two high-order bytes of the source-register are ignored. The target-field has to be a halfword (->> boundary!) in storage. The source-register remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example



STM Store multiple

Mnemonic	Operands
STM	R1 , R3 , D2 (B2)

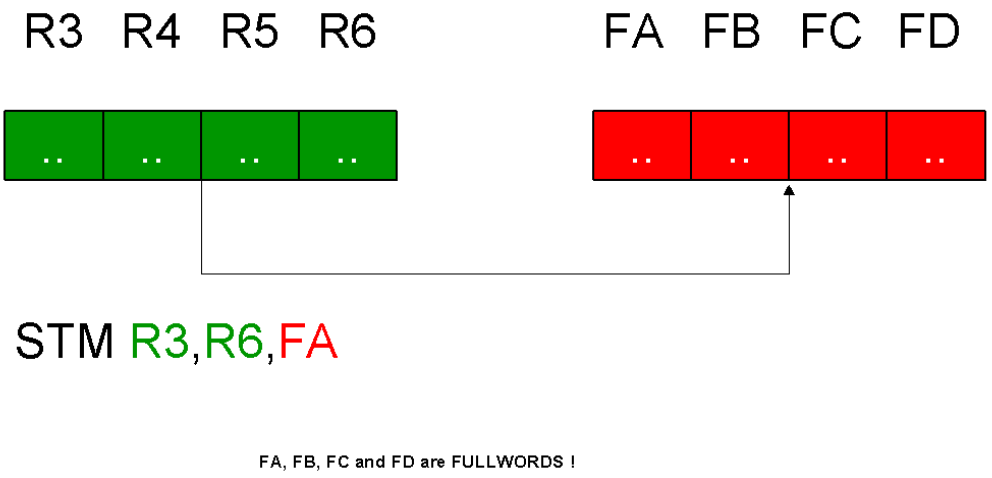
Machine format (RS)

9 0	R1	R3	B2	D2
0	8	12	16	20 31

To move the binary data from adjacent registers to a storage location, the STM (STore Multiple) instruction is performed. The contents of the source-registers, which are defined by the starting-register (1st operand) and the ending-register (3rd operand), are stored in the target-field (2nd operand). The source-registers are stored in the ascending order of their register numbers (with register R0 follows register R15). The target-field has to be a sequence of fullwords (->> boundary!) in storage. The number of source-registers defines the length of the target-field. The source-registers remain unchanged.

Condition codes
Condition code remains unchanged!

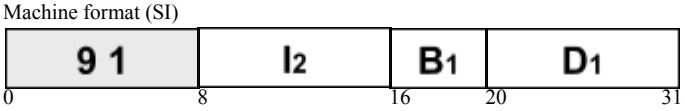
Graphical example



TM Test under mask

Mnemonic	Operands
TM	D_1 (B_1) , I_2

Machine format (SI)



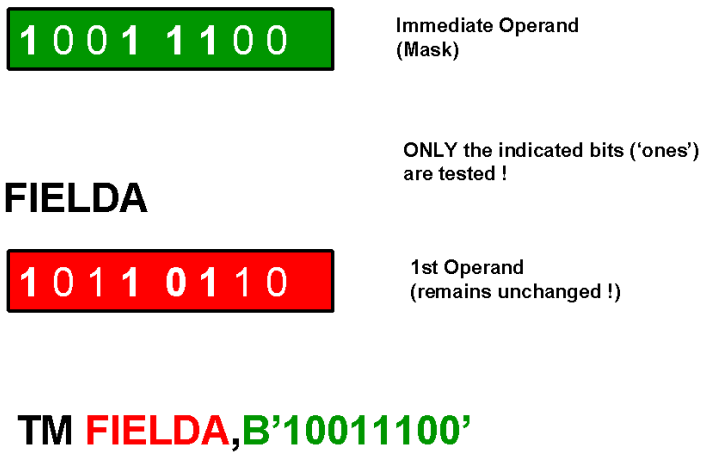
To test the bits of a single byte of data, whether they are on or off, the TM (Test under Mask) instruction is performed. The bits of the 1st-operand field are selected and tested by an eight-bit mask, given by the 2nd-operand. A mask-bit of 'one' indicates, that the corresponding storage-bit is to be tested. If the mask-bit is zero, the storage-bit is ignored.
The 2nd operand is also called the immediate operand. The immediate data (mask), defined at program coding, is contained within the instruction itself. The length of the 1st-operand field is not of interest. Only the 1st byte of the field is tested. The result is indicated in the condition code which may be checked subsequently.
The 1st-operand field remains unchanged.

1)* all selected bits are zeros
2)* selected bits are mixed
3)* all selected bits are ones

Condition codes

0 all selected bits are zero
1 selected bits are mixed
2 **not used!**
3 all selected bits are one

Graphical example



TR Translate

Mnemonic | Operands

TR	$D_1 (L_1 , B_1) , D_2 (B_2)$

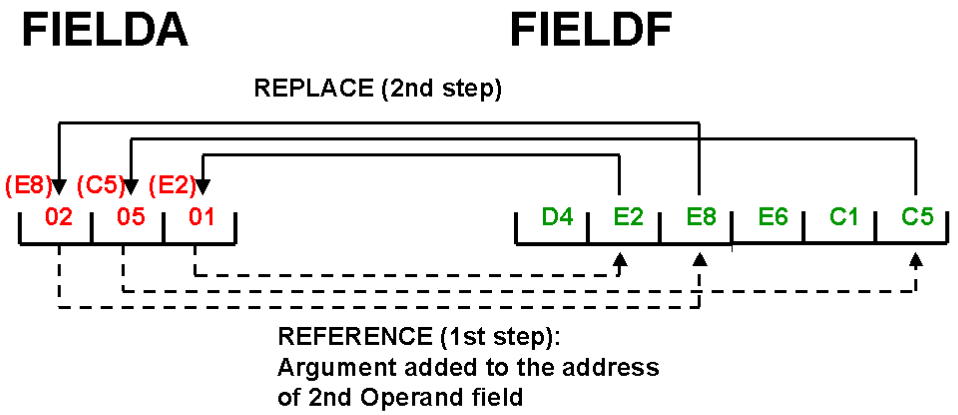
Machine format (SS1)

D C	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To translate a hexadecimal value in a field to printable characters, the TR (TRanslate) instruction is performed. Each byte of the 1st-operand field is used as an eight-bit argument to reference a function byte within the 2nd-operand field. First the contents of the argument-byte is added to the initial 2nd-operand address to point at a function-byte. After that the found function-byte replaces the active argument-byte in the 1st-operand field.
The instruction is processed from left to right, byte by byte.
See also the TRT instruction.

Condition codes
Condition code remains unchanged!

Graphical example



TR FIELDA,FIELDF

TRT Translate and test

Mnemonic | Operands

TRT	$D_1 (L_1 , B_1) , D_2 (B_2)$

Machine format (SS1)

D D	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

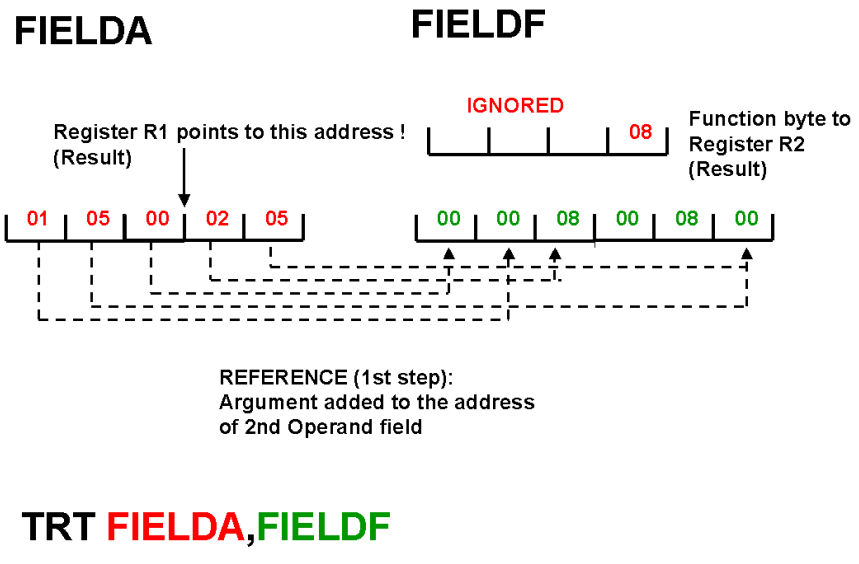
To check a numeric, an alpha or an alphanumeric field for right characters, the TRT (TRanslate and Test) instruction is performed. Each byte of the 1st-operand field is used as an eight-bit argument to reference a function byte within the 2nd-operand field. The contents of the argument-byte is added to the initial 2nd-operand address to point at a function-byte. The processing stops immediately if the found function-byte is not zero (X'00') or the end of the 1st-operand field is reached. The found non-zero function-byte is inserted in the rightmost byte of the register R2 and additionally the related argument-byte address (within the 1st-operand field) is placed into the register R1. If all found function-bytes are zero, the contents of registers R1 and R2 remain unchanged. The result of the instruction is also indicated in the condition code which may be checked subsequently.
The instruction is processed from left to right, byte by byte.
See also the TR instruction.

1)* all function bytes found are zero
2)* a nonzero function byte is found
3)* a nonzero function byte is found and last byte 1st opnd

Condition codes

0 all function bytes found are zero
1 a nonzero function byte is found
2 a nonzero function byte is found and it's the last byte of the 1st operand field
3 **not used!**

Graphical example



TRT FIELDA,FIELDF

UNPK Unpack

Mnemonic | Operands

UNPK	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

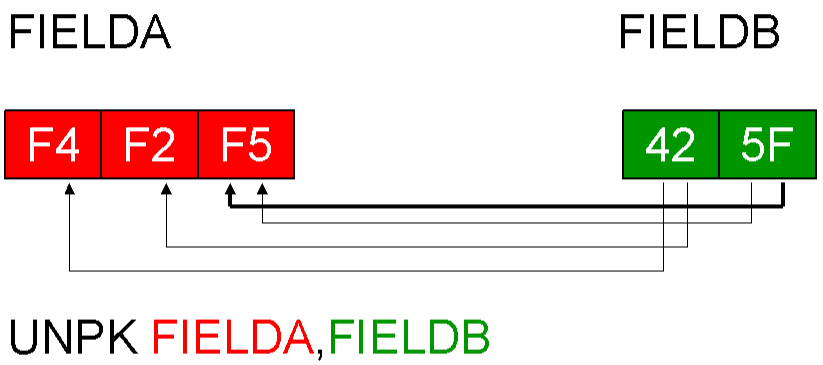
Machine format (SS2)

F 3	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

The UNPK (UNPacK) instruction converts packed decimal data to zoned format. The packed decimal data of the source-field (2nd operand) is converted to zoned-format data and placed in the target-field (1st operand). The assembler must know the length of each operand field, which may be up to 16 (!) bytes long. Execution proceeds from right to left.
If the length of the 1st-operand (receiving) field is too small, the leftmost positions of the 2nd-operand (sending) field are truncated. If the length of the receiving field is too large, decimal zeros are added. The source-field remains unchanged.

Condition codes
Condition code remains unchanged!

Graphical example

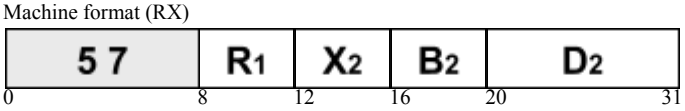


The target field will be left padded with decimal zeroes !

X Exclusive or fullword

Mnemonic	Operands
X	R_1 , D_2 (X_2 , B_2)

Machine format (RX)

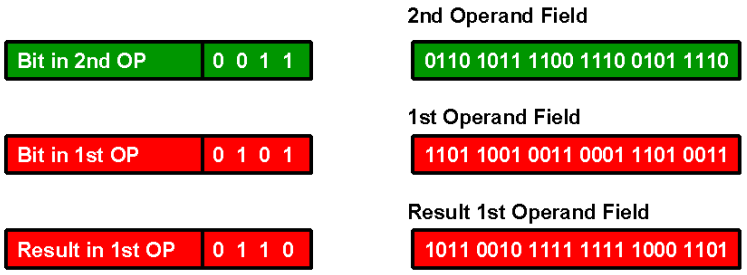


To set single bits of a register to 'one' or to 'zero' in one step, the X (eXclusive or fullword) instruction is performed. The 'one'-bits of the 1st-operand register are changed to 'zero', respectively the 'zero'-bits are changed to 'one', if the corresponding bits of the 2nd-operand field, a fullword (->> boundary!) in storage, are 'one'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the XR instruction.ain unchanged.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	

Graphical example



XC Exclusive or character

Mnemonic | Operands

XC	$D_1 (L_1 , B_1) , D_2 (B_2)$

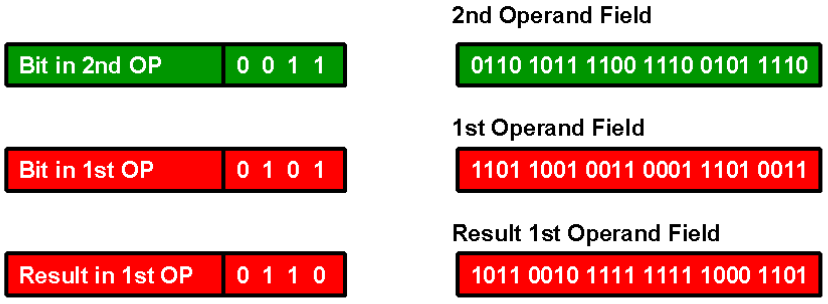
Machine format (SS1)

D 7	L_1	B_1	D_1	B_2	D_2
0	8	16	20	32	36 47

To set single bits of a storage field to 'one' or 'zero' in one step, the XC (eXclusive or Character) instruction is performed. The 'one'-bits of the 1st-operand field are changed to 'zero', respectively the 'zero'-bits are changed to 'one', if the corresponding bits of the 2nd-operand field are 'one'. In all other cases the bits of the 1st-operand field remain unchanged.
The result is indicated in the condition code which may be checked subsequently.

Condition codes

0	**= 0**	1st operand field is zero
1	**!= 0**	1st operand field is not zero
2	**not used!**	
3	**not used!**	

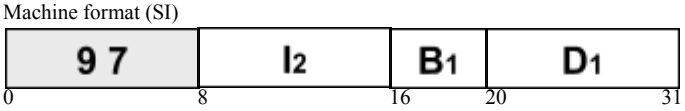


Graphical example

XI Exclusive or immediate

Mnemonic	Operands
XI	D_1 (B_1) , I_2

Machine format (SI)

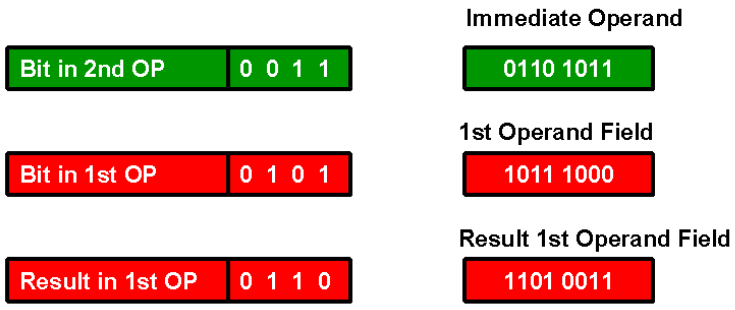


To set the bits of a single byte of data to 'one' or to 'zero' in one step, the XI (eXclusive or Immediate) instruction is performed. The 'one'-bits of the 1st-operand field are changed to 'zero', respectively the 'zero'-bits are changed to 'one', if the corresponding bits of the 2nd-operand field are 'one'. In all other cases the bits of the 1st-operand field remain unchanged.
The 2nd operand is also called the immediate operand. The immediate data, defined at program coding, is contained within the instruction itself. The length of the 1st-operand field is not of interest. Only the 1st byte of the field is considered. The result is indicated in the condition code which may be checked subsequently.

Condition codes

0	**= 0**	1st operand byte is zero
1	**!= 0**	1st operand byte is not zero
2	**not used!**	
3	**not used!**	

Graphical example



XR Exclusive or register

Mnemonic | Operands

XR	R_1 , R_2

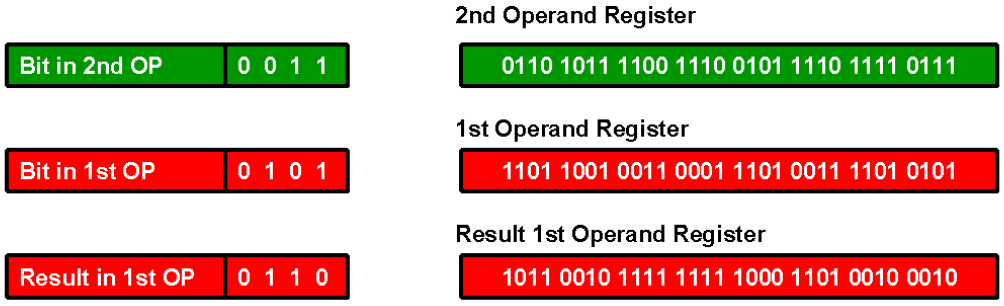
Machine format (RR)

1 7	R_1	R_2
0	8	12 15

To set single bits of a register to 'one' or 'zero' in one step, the XR (eXclusive or Register) instruction is performed. The 'one'-bits of the 1st-operand register are changed to 'zero', respectively the 'zero'-bits are changed to 'one', if the corresponding bits of the 2nd-operand register are 'one'. In all other cases the bits of the 1st-operand register remain unchanged.
The result is indicated in the condition code which may be checked subsequently.
See also the X instruction.

Condition codes

0	**= 0**	1st operand register is zero
1	**!= 0**	1st operand register is not zero
2	**not used!**	
3	**not used!**	



Graphical example

ZAP Zero and add packed decimal data

Mnemonic | Operands

ZAP	$D_1 (L_1 , B_1) , D_2 (L_2 , B_2)$

Machine format (SS2)

F 8	L_1	L_2	B_1	D_1	B_2	D_2
0	8	12	16	20	32	36 47

To set up packed decimal data in an other field, the ZAP (Zero and Add Packed decimal data) instruction is performed. First the 1st-operand field is set to packed decimal zero. Then the contents of the 2nd-operand field is added to the 1st-operand zero value. The initial contents of the 2nd-operand field has to be in correct packed decimal format, otherwise the instruction results in a system error (Data Exception). The result is also in packed decimal format. The 2nd-operand field remains unchanged.
The calculation follows algebraic rules. The result is tested and a condition code is set. In case of an overflow the overflow digits are ignored and additionally the condition code is set to 3

Condition codes

0	**= 0**	result is zero
1	**< 0**	result is negative
2	**> 0**	result is positive
3	**overflow**	result doesn't fit into the 1st operand field

Graphical example